

ANEXO A: DECISIONES DE USO DE COLUMNAS

tabla,columna,existe,decision,motivo

players,player_id,True,auxiliar,Clave primaria

players,name,True,auxiliar,"Trazabilidad, no predictor"

players,date_of_birth,True,usar,Base para calcular la edad a fecha de corte

players,position,True,usar,Predictor y criterio de exclusión de porteros

players,sub_position,True,usar,Predictor: posición específica

players,foot,True,usar,Predictor: pie dominante

players,height_in_cm,True,usar,Predictor: perfil físico

players,country_of_citizenship,True,usar,Predictor: nacionalidad

players,market_value_in_eur,True,descartar,"Valor actual, contemporáneo al objetivo"

players,highest_market_value_in_eur,True,descartar,Máximo histórico posterior a la fecha de corte

players,current_club_id,True,descartar,"Club actual, no el de la temporada analizada"

players,contract_expiration_date,True,descartar,"Contrato vigente hoy, no el de la temporada"

players,agent_name,True,descartar,Alta cardinalidad y sin valor predictivo

players,image_url,True,descartar,Sin valor predictivo

players,url,True,descartar,Sin valor predictivo

players,city_of_birth,True,descartar,No contemplada en el alcance

players,country_of_birth,True,descartar,No contemplada en el alcance

players,current_club_domestic_competition_id,True,descartar,No contemplada en el alcance

players,current_club_name,True,descartar,No contemplada en el alcance

players,current_national_team_id,True,descartar,No contemplada en el alcance

players,first_name,True,descartar,No contemplada en el alcance

players,international_caps,True,descartar,No contemplada en el alcance

players,international_goals,True,descartar,No contemplada en el alcance

players,last_name,True,descartar,No contemplada en el alcance

players,last_season,True,descartar,No contemplada en el alcance

players,player_code,True,descartar,No contemplada en el alcance

player_valuations,player_id,True,auxiliar,Clave foránea

player_valuations,date,True,usar,Define la ventana temporal del objetivo

player_valuations,market_value_in_eur,True,usar,Variable objetivo

player_valuations,current_club_id,True,auxiliar,Club en la fecha de la valoración

player_valuations,current_club_name,True,descartar,No contemplada en el alcance

player_valuations,player_club_domestic_competition_id,True,descartar,No contemplada en el alcance

appearances,appearance_id,True,auxiliar,Clave primaria

appearances,game_id,True,auxiliar,Clave para unir con games

appearances,player_id,True,auxiliar,Clave de agrupación

appearances,player_club_id,True,usar,Permite deducir el club de la temporada

appearances,competition_id,True,usar,Tipo y nivel de competición

appearances,date,True,auxiliar,Coherencia temporal

appearances,minutes_played,True,usar,Base de todas las métricas por 90 minutos

appearances,goals,True,usar,Producción ofensiva

appearances,assists,True,usar,Producción ofensiva

appearances,yellow_cards,True,usar,Disciplina

appearances,red_cards,True,usar,Disciplina

appearances,player_name,True,descartar,Duplica un identificador ya presente

appearances,player_current_club_id,True,descartar,No contemplada en el alcance

clubs,club_id,True,auxiliar,Clave primaria

clubs,domestic_competition_id,True,usar,Liga del club

clubs,stadium_seats,True,usar,Proxy estable de dimensión del club

clubs,total_market_value,True,descartar,"Valor actual de la plantilla, no el de la temporada"

clubs,squad_size,True,descartar,Tamaño actual de la plantilla

clubs,average_age,True,descartar,Edad media actual de la plantilla

clubs,net_transfer_record,True,descartar,Saldo de fichajes actual

clubs,coach_name,True,descartar,"Entrenador actual, sin valor predictivo"

clubs,club_code,True,descartar,No contemplada en el alcance

clubs,filename,True,descartar,No contemplada en el alcance

clubs,foreigners_number,True,descartar,No contemplada en el alcance

clubs,foreigners_percentage,True,descartar,No contemplada en el alcance

clubs,last_season,True,descartar,No contemplada en el alcance

clubs,name,True,descartar,No contemplada en el alcance

clubs,national_team_players,True,descartar,No contemplada en el alcance

clubs,stadium_name,True,descartar,No contemplada en el alcance

clubs,url,True,descartar,No contemplada en el alcance

games,game_id,True,auxiliar,Clave primaria

games,season,True,usar,Asigna cada aparición a su temporada

games,competition_id,True,auxiliar,Competición del partido

games,date,True,auxiliar,Coherencia temporal

games,home_club_id,True,auxiliar,Verificación de consistencia

games,away_club_id,True,auxiliar,Verificación de consistencia

games,aggregate,True,descartar,No contemplada en el alcance

games,attendance,True,descartar,No contemplada en el alcance

games,away_club_formation,True,descartar,No contemplada en el alcance

games,away_club_goals,True,descartar,No contemplada en el alcance
games,away_club_manager_name,True,descartar,No contemplada en el alcance
games,away_club_name,True,descartar,No contemplada en el alcance
games,away_club_position,True,descartar,No contemplada en el alcance
games,competition_type,True,descartar,No contemplada en el alcance
games,home_club_formation,True,descartar,No contemplada en el alcance
games,home_club_goals,True,descartar,No contemplada en el alcance
games,home_club_manager_name,True,descartar,No contemplada en el alcance
games,home_club_name,True,descartar,No contemplada en el alcance
games,home_club_position,True,descartar,No contemplada en el alcance
games,referee,True,descartar,No contemplada en el alcance
games,round,True,descartar,No contemplada en el alcance
games,stadium,True,descartar,No contemplada en el alcance
games,url,True,descartar,No contemplada en el alcance
club_games,club_id,True,auxiliar,Clave
club_games,game_id,True,auxiliar,Clave
club_games,own_goals,True,usar,Rendimiento del club: goles a favor
club_games,opponent_goals,True,usar,Rendimiento del club: goles en contra
club_games,is_win,True,usar,Rendimiento del club: puntos por partido
club_games,hosting,True,descartar,No contemplada en el alcance
club_games,opponent_id,True,descartar,No contemplada en el alcance
club_games,opponent_manager_name,True,descartar,No contemplada en el alcance
club_games,opponent_position,True,descartar,No contemplada en el alcance
club_games,own_manager_name,True,descartar,No contemplada en el alcance
club_games,own_position,True,descartar,No contemplada en el alcance
game_lineups,game_id,True,auxiliar,Clave
game_lineups,player_id,True,auxiliar,Clave
game_lineups,type,True,usar,Titular o suplente
game_lineups,team_captain,True,usar,Rol de liderazgo
game_lineups,club_id,True,descartar,No contemplada en el alcance
game_lineups,date,True,descartar,No contemplada en el alcance
game_lineups,game_lineups_id,True,descartar,No contemplada en el alcance
game_lineups,number,True,descartar,No contemplada en el alcance
game_lineups,player_name,True,descartar,No contemplada en el alcance
game_lineups,position,True,descartar,No contemplada en el alcance
competitions,competition_id,True,auxiliar,Clave primaria
competitions,type,True,usar,"Liga doméstica, copa o internacional"
competitions,name,True,auxiliar,Legibilidad
competitions,country_name,True,auxiliar,Geografía
competitions,competition_code,True,descartar,No contemplada en el alcance

competitions,confederation,True,descartar,No contemplada en el alcance
competitions,country_id,True,descartar,No contemplada en el alcance
competitions,domestic_league_code,True,descartar,No contemplada en el alcance
competitions,sub_type,True,descartar,No contemplada en el alcance
competitions,total_clubs,True,descartar,No contemplada en el alcance
competitions,url,True,descartar,No contemplada en el alcance

ANEXO B: AUDITORÍA DE CONSISTENCIA INTERNA

comprobación,casos,%
minutes_played negativo,0,0.0
minutes_played > 120,3,0.0
goals negativo,0,0.0
yellow_cards > 2 en un partido,0,0.0
red_cards > 1 en un partido,0,0.0
fecha de nacimiento nula o ilegible,49,0.098
nacimiento fuera de 1950-2010,1,0.002
altura fuera de 150-215 cm,13,0.026
valor de mercado <= 0,0,0.0
partido fuera del rango de su temporada,1940,2.181
appearance_id duplicado,0,0.0
mismo jugador dos veces en el mismo partido,0,0.0
misma valoración duplicada (jugador + fecha),0,0.0
club del jugador ajeno al partido,0,0.0

ANEXO C: CATÁLOGO DE VARIABLES

variable, bloque, descripción, origen y cálculo, en_dataset

edad, Perfil, Edad del jugador a la fecha de corte, $(\text{fecha_corte} - \text{date_of_birth}) / 365,25$, con fecha_corte = 1 de mayo de S+1", True

height_in_cm, Perfil, Altura en centímetros, players.height_in_cm, True

position, Perfil, "Posición general (ataque, medio, defensa)", players.position, True

sub_position, Perfil, Posición específica, players.sub_position, True

foot, Perfil, Pie dominante, players.foot; nulos como categoría propia, True

partidos, Producción, Partidos disputados en la temporada, Recuento de game_id distintos en appearances, True

minutos, Producción, Minutos totales jugados, Suma de minutes_played, True

goles, Producción, Goles marcados, Suma de goals, True

asistencias, Producción, Asistencias, Suma de assists, True

amarillas, Producción, Tarjetas amarillas, Suma de yellow_cards, True

rojas, Producción, Expulsiones, Suma de red_cards, True

goles_90, Eficiencia, Goles por cada 90 minutos jugados, $\text{goles} / (\text{minutos} / 90)$, True

asistencias_90, Eficiencia, Asistencias por 90 minutos, $\text{asistencias} / (\text{minutos} / 90)$, True

ga_90, Eficiencia, Contribución de gol por 90 minutos, $(\text{goles} + \text{asistencias}) / (\text{minutos} / 90)$, True

amarillas_90, Eficiencia, Amarillas por 90 minutos, $\text{amarillas} / (\text{minutos} / 90)$, True

min_por_partido, Rol, Minutos medios por partido disputado, $\text{minutos} / \text{partidos}$, True

ratio_titular, Rol, Proporción de convocatorias como titular, "titularidades / convocatorias, desde game_lineups", True

capitanias, Rol, Partidos disputados como capitán, Recuento de team_captain en game_lineups, True

partidos_liga, Competición, Partidos en competición doméstica, Recuento de apariciones con type = domestic_league, True

partidos_euro, Competición, Partidos en competición europea, "Recuento de apariciones en CL, EL o UCOL", True

n_competiciones, Competición, Competiciones distintas disputadas, Recuento de competition_id distintos, True

jugo_europa, Competición, Indicador de participación europea, 1 si partidos_euro > 0, True

club_ppp, Club, Puntos por partido del club en su liga, $(\text{victorias} \times 3 + \text{empates}) / \text{partidos}$, desde club_games", True

club_dif_g, Club, Diferencia de goles por partido del club, $(\text{goles a favor} - \text{en contra}) / \text{partidos}$, desde club_games", True

club_rank, Club, Posición del club en su liga, Ranking por puntos dentro de competición y temporada, True

club_valor_medio, Club, Valor medio de la plantilla del club, Media de mv_prev de los jugadores del club en esa temporada, True

club_aforo, Club, Capacidad del estadio, "clubs.stadium_seats; foto actual, admitida por su estabilidad", True

liga, Club, Competición doméstica del jugador, Liga en la que acumuló más minutos, True

log_mv_prev, Historial, "Valoración previa, deflactada y en logaritmo", $\ln(\text{mv_prev}) - \ln(\text{índice})$

ANEXO D: CÓDIGO DE NOTEBOOK 01_DATOS.IPYNB

Estimación del valor de mercado de futbolistas profesionales

Notebook 1 — Adquisición, auditoría y construcción del dataset analítico

****Trabajo Fin de Máster**** · Máster en Big Data, Data Science e Inteligencia Artificial · Universidad Complutense de Madrid

Qué hace este notebook

```
|||
|---|---|
| **Consume** | Los 12 ficheros CSV del conjunto *Football Data from Transfermarkt* |
| **Produce** | `data/processed/dataset_analitico.parquet` |
| **Siguiente** | `02_modelos.ipynb`, que parte del parquet generado aquí |
```

Estructura

1. ****Configuración y adquisición**** — descarga reproducible y congelación de una instantánea
2. ****Comprensión de las tablas**** — diccionario de datos y decisión sobre cada columna
3. ****Auditoría de calidad**** — integridad, consistencia y cobertura de los datos en crudo
4. ****Construcción del dataset analítico**** — de 1,9 M de apariciones a una fila por jugador y temporada
5. ****Verificación de salida**** — comprobaciones sobre el conjunto generado

Nota metodológica

Este notebook ****no contiene análisis exploratorio****. La exploración se realiza sobre el dataset analítico ya construido y se recoge en el notebook 2, puesto que la unidad de análisis del estudio es el jugador-temporada y esa unidad no existe en los datos en crudo.

Lo que sí se realiza aquí es una ****auditoría de calidad****, que responde a una pregunta distinta: si los datos de partida soportan la transformación que se pretende aplicar.

1. Configuración y adquisición

1.1. Configuración del entorno

Se fija la semilla de aleatoriedad, se definen las rutas del proyecto mediante `pathlib` y se registran las versiones de las librerías empleadas, garantizando así su reproducibilidad.

```
from pathlib import Path
```

```
import random
```

```
import numpy as np
```

```
import pandas as pd
```

--- Reproducibilidad -----

```

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# --- Rutas del proyecto -----
PROJ = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

RAW    = PROJ / "data" / "raw"
INTERIM = PROJ / "data" / "interim"
PROCESSED = PROJ / "data" / "processed"
MODELS  = PROJ / "models"
FIGURES = PROJ / "reports" / "figures"

for p in (RAW, INTERIM, PROCESSED, MODELS, FIGURES):
    p.mkdir(parents=True, exist_ok=True)

# --- Presentación -----
pd.set_option("display.max_columns", 60)
pd.set_option("display.width", 140)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")

print("Proyecto:", PROJ)
print("Datos en:", RAW)
# Versiones del entorno (evidencia de reproducibilidad para el Anexo)
%load_ext watermark
%watermark -v -m -p pandas,numpy,pyarrow
## 1.2. Descarga y congelación de la instantánea

```

Este conjunto de datos se actualiza **semanalmente**, por lo que si el análisis siempre leyera directamente de la última versión disponible, los resultados dejarían de ser reproducibles: las valoraciones cambiarían y las cifras de la memoria no coincidirían con una reejecución posterior.

Para evitarlo se descarga una única vez mediante `kagglehub` y se **congela** una instantánea fechada en `data/raw/`.

```

import shutil
from datetime import date
import kagglehub

DATASET = "davidcariboo/player-scores"
SNAPSHOT = RAW / f"snapshot_{date.today():%Y%m%d}"

existentes = sorted(RAW.glob("snapshot_*"))

if existentes:
    DATA = existentes[-1]
    print(f"Ya existe una instantánea: {DATA.name}. No se descarga de nuevo.")
else:
    print("Descargando desde Kaggle...")
    origen = Path(kagglehub.dataset_download(DATASET))

```

```

SNAPSHOT.mkdir(parents=True)
n = 0
for csv in origen.rglob("*.csv"):
    shutil.copy2(csv, SNAPSHOT / csv.name)
    n += 1

(SNAPSHOT / "_METADATA.txt").write_text(
    f"dataset: {DATASET}\n"
    f"fecha_descarga: {date.today():%Y-%m-%d}\n"
    f"origen_cache: {origen}\n"
    f"ficheros_csv: {n}\n",
    encoding="utf-8",
)
DATA = SNAPSHOT
print(f"Copiados {n} ficheros CSV a {DATA}")

print("\n--- Instantánea de trabajo ---")
print((DATA / "_METADATA.txt").read_text(encoding="utf-8"))
## 1.3. Inventario de ficheros

```

Volumetría del conjunto de partida. El recuento de filas se realiza sin cargar los ficheros en memoria, leyéndolos línea a línea, para solo cargar aquellos con los que realmente vamos a trabajar.

```

def contar_filas(ruta):
    with open(ruta, "r", encoding="utf-8") as f:
        return sum(1 for _ in f) - 1

inventario = pd.DataFrame([
    {"tabla": f.stem,
     "filas": contar_filas(f)}
    for f in sorted(DATA.glob("*.csv"))
]).sort_values("filas", ascending=False, ignore_index=True)

print(f"Total: {inventario['filas'].sum():,} filas")
inventario

```

2. Comprensión de las tablas

2.1. Diccionario de tablas

El conjunto está organizado como un **modelo relacional de 12 tablas** con distintos niveles de granularidad. Comprender qué representa cada fila en cada tabla es imprescindible antes de cualquier transformación, ya que determina si una tabla debe **agregarse** o simplemente **unirse**.

Tabla	Una fila representa	Uso en este trabajo
--- --- ---		
`competitions`	Una competición	Filtro de ligas y tipo de torneo
`games`	Un partido	Aporta la temporada de cada partido
`clubs`	Un club	Contexto (aforo)
`players`	Un jugador	Perfil
`player_valuations`	Una revisión del valor de un jugador	**Variable objetivo**

- | `appearances` | Un jugador en un partido | Núcleo del rendimiento |
- | `game\_events` | Un evento dentro de un partido | Fuera del alcance |
- | `game\_lineups` | Un jugador en la alineación de un partido | Titularidades y capitanía |
- | `club\_games` | Un club en un partido | Forma del club por temporada |
- | `transfers` | Un traspaso | Fuera del alcance |
- | `countries` | Un país | Fuera del alcance |
- | `player\_teammates\_played\_with` | Coincidencias entre jugadores | Fuera del alcance |

****Anotación:**** `appearances` debe agregarse (un jugador con 34 partidos ocupa 34 filas y debe quedar en una), mientras que `players` o `clubs` se unen como tablas de referencia, ya que tienen una fila por entidad.

2.2. Esquemas reales de las tablas

```
for f in sorted(DATA.glob("*.csv")):
    cols = pd.read_csv(f, nrows=100, encoding="utf-8", low_memory=False).columns
    print(f"\n=== {f.stem} ({len(cols)} columnas) ===")
    print(", ".join(cols))
```

2.3. Carga de las tablas del alcance

De las 12 tablas disponibles se cargan las 9 que intervienen en el estudio. Se descartan `game\_events` (granularidad superior a la necesaria), `countries` y `player\_teammates\_played\_with` (sin aportación al objetivo).

```
TABLAS = ["appearances", "player_valuations", "players", "clubs",
          "competitions", "games", "game_lineups", "club_games"]
```

```
df = {}
for t in TABLAS:
    ruta = DATA / f"{t}.csv"
    if not ruta.exists():
        print(f"AVISO: no existe {t}.csv")
        continue
    df[t] = pd.read_csv(ruta, encoding="utf-8", low_memory=False)
    print(f"{t:<20} {df[t].shape[0]:>10,} x {df[t].shape[1]:>3}")
```

2.4. Decisión sobre cada columna

Antes de transformar nada se revisa columna a columna y se documenta una decisión justificada.

El catálogo completo se exporta al ****Anexo A****.

Decisiones por columna: usar / auxiliar / descartar, con su motivo.

```
DECISIONES = {
    "players": {
        "player_id": ("auxiliar", "Clave primaria"),
        "name": ("auxiliar", "Trazabilidad, no predictor"),
        "date_of_birth": ("usar", "Base para calcular la edad a fecha de corte"),
        "position": ("usar", "Predictor y criterio de exclusión de porteros"),
        "sub_position": ("usar", "Predictor: posición específica"),
        "foot": ("usar", "Predictor: pie dominante"),
        "height_in_cm": ("usar", "Predictor: perfil físico"),
        "country_of_citizenship": ("usar", "Predictor: nacionalidad"),
        "market_value_in_eur": ("descartar", "Valor actual, contemporáneo al objetivo"),
        "highest_market_value_in_eur": ("descartar", "Máximo histórico posterior a la fecha de corte"),
    }
```

```

    "current_club_id":      ("descartar", "Club actual, no el de la temporada analizada"),
    "contract_expiration_date": ("descartar", "Contrato vigente hoy, no el de la temporada"),
    "agent_name":          ("descartar", "Alta cardinalidad y sin valor predictivo"),
    "image_url":           ("descartar", "Sin valor predictivo"),
    "url":                  ("descartar", "Sin valor predictivo"),
  },
  "player_valuations": {
    "player_id":           ("auxiliar", "Clave foránea"),
    "date":                ("usar", "Define la ventana temporal del objetivo"),
    "market_value_in_eur": ("usar", "Variable objetivo"),
    "current_club_id":     ("auxiliar", "Club en la fecha de la valoración"),
  },
  "appearances": {
    "appearance_id":       ("auxiliar", "Clave primaria"),
    "game_id":             ("auxiliar", "Clave para unir con games"),
    "player_id":           ("auxiliar", "Clave de agrupación"),
    "player_club_id":      ("usar", "Permite deducir el club de la temporada"),
    "competition_id":      ("usar", "Tipo y nivel de competición"),
    "date":                ("auxiliar", "Coherencia temporal"),
    "minutes_played":      ("usar", "Base de todas las métricas por 90 minutos"),
    "goals":               ("usar", "Producción ofensiva"),
    "assists":             ("usar", "Producción ofensiva"),
    "yellow_cards":        ("usar", "Disciplina"),
    "red_cards":           ("usar", "Disciplina"),
    "player_name":         ("descartar", "Duplica un identificador ya presente"),
  },
  "clubs": {
    "club_id":             ("auxiliar", "Clave primaria"),
    "domestic_competition_id": ("usar", "Liga del club"),
    "stadium_seats":       ("usar", "Proxy estable de dimensión del club"),
    "total_market_value":  ("descartar", "Valor actual de la plantilla, no el de la temporada"),
    "squad_size":          ("descartar", "Tamaño actual de la plantilla"),
    "average_age":         ("descartar", "Edad media actual de la plantilla"),
    "net_transfer_record": ("descartar", "Saldo de fichajes actual"),
    "coach_name":          ("descartar", "Entrenador actual, sin valor predictivo"),
  },
  "games": {
    "game_id":             ("auxiliar", "Clave primaria"),
    "season":              ("usar", "Asigna cada aparición a su temporada"),
    "competition_id":      ("auxiliar", "Competición del partido"),
    "date":                ("auxiliar", "Coherencia temporal"),
    "home_club_id":        ("auxiliar", "Verificación de consistencia"),
    "away_club_id":        ("auxiliar", "Verificación de consistencia"),
  },
  "club_games": {
    "club_id":             ("auxiliar", "Clave"),
    "game_id":             ("auxiliar", "Clave"),
    "own_goals":           ("usar", "Rendimiento del club: goles a favor"),
    "opponent_goals":      ("usar", "Rendimiento del club: goles en contra"),
    "is_win":              ("usar", "Rendimiento del club: puntos por partido"),
  },

```

```

"game_lineups": {
    "game_id":          ("auxiliar", "Clave"),
    "player_id":        ("auxiliar", "Clave"),
    "type":              ("usar",    "Titular o suplente"),
    "team_captain":      ("usar",    "Rol de liderazgo"),
},
"competitions": {
    "competition_id":    ("auxiliar", "Clave primaria"),
    "type":              ("usar",    "Liga doméstica, copa o internacional"),
    "name":              ("auxiliar", "Legibilidad"),
    "country_name":      ("auxiliar", "Geografía"),
},
}

filas = []
for tabla, cols in DECISIONES.items():
    if tabla not in df:
        continue
    presentes = set(df[tabla].columns)
    for col, (decision, motivo) in cols.items():
        filas.append({"tabla": tabla, "columna": col,
                      "existe": col in presentes,
                      "decision": decision, "motivo": motivo})
    # Columnas presentes en el fichero pero no contempladas
    for col in sorted(presentes - set(cols)):
        filas.append({"tabla": tabla, "columna": col, "existe": True,
                      "decision": "descartar", "motivo": "No contemplada en el alcance"})

catalogo = pd.DataFrame(filas)
catalogo.to_csv(PROCESSED / "anexoA_decisiones_columnas.csv",
                index=False, encoding="utf-8")

print("Resumen de decisiones:")
print(catalogo["decision"].value_counts().to_string())
print("\nColumnas declaradas que NO existen en el fichero (revisar):")
faltan = catalogo[~catalogo["existe"]]
print(faltan[["tabla", "columna"]].to_string(index=False) if len(faltan) else " ninguna")

catalogo[catalogo["decision"] == "descartar"].head(20)
---
```

3. Auditoría de calidad

La auditoría responde a una única pregunta: ****¿soportan estos datos la transformación que se pretende aplicar?****

Se examinan seis dimensiones: integridad de la adquisición, estructura y tipos, integridad referencial, completitud, cobertura y consistencia interna.

3.1. Completitud

```

print("\n=== COLUMNAS CON AUSENCIAS RELEVANTES (> 5%) ===")
filas = []
for t in TABLAS:
```

```

if t not in df:
    continue
n = df[t].isna().mean()
for col, pct in n[n > 0.05].items():
    filas.append({"tabla": t, "columna": col, "nulos_%": round(100*pct, 1)})
resumen = pd.DataFrame(filas).sort_values("nulos_%", ascending=False)
print(resumen.to_string(index=False) if len(resumen) else " ninguna")

print("\n=== COLUMNAS CLAVE: ausencias esperadas cero ===")
CLAVES = [("appearances", "player_id"), ("appearances", "game_id"),
          ("appearances", "minutes_played"), ("games", "season"),
          ("games", "game_id"), ("player_valuations", "player_id"),
          ("player_valuations", "market_value_in_eur"),
          ("club_games", "club_id"), ("club_games", "is_win"),
          ("clubs", "domestic_competition_id")]
for t, c in CLAVES:
    if t in df and c in df[t].columns:
        k = df[t][c].isna().sum()
        print(f" {t}. {c}:<28 {k:>7,} nulos { 'OK' if k == 0 else '<-- REVISAR' }")
## 3.2. Integridad referencial

```

Se verifica que las claves foráneas de cada tabla tienen correspondencia en su tabla de referencia.

El criterio no es que el resultado sea cero, sino conocer la magnitud y haber decidido qué hacer con ella.

```
def huerfanas(hija, col_h, madre, col_m):
```

```

    izq = set(df[hija][col_h].dropna().unique())
    der = set(df[madre][col_m].dropna().unique())
    faltan = izq - der

    filas = df[hija][col_h].isin(faltan).mean()
    return {"relación": f"{hija}.{col_h} → {madre}.{col_m}",
            "claves": len(izq), "huérfanas": len(faltan),
            "%_claves": round(100 * len(faltan) / max(len(izq), 1), 2),
            "%_filas": round(100 * filas, 2)}

```

```

checks = [
    ("appearances", "player_id", "players", "player_id"),
    ("appearances", "game_id", "games", "game_id"),
    ("appearances", "player_club_id", "clubs", "club_id"),
    ("player_valuations", "player_id", "players", "player_id"),
    ("games", "competition_id", "competitions", "competition_id"),
    ("games", "home_club_id", "clubs", "club_id"),
    ("club_games", "game_id", "games", "game_id"),
    ("club_games", "club_id", "clubs", "club_id"),
    ("game_lineups", "game_id", "games", "game_id"),
    ("game_lineups", "player_id", "players", "player_id"),
]

```

```

integridad = pd.DataFrame([huerfanas(*c) for c in checks
                           if c[0] in df and c[2] in df])
integridad.to_csv(PROCESSED / "anexoB_integridad.csv", index=False, encoding="utf-8")

print("=== INTEGRIDAD REFERENCIAL ===")
print(integridad.to_string(index=False))

# RESULTADO
graves = integridad[integridad["%_filas"] > 5]
print(f"\nRelaciones con mas del 5% de FILAS afectadas: "
      f"{len(graves)} de {len(integridad)}")
if len(graves):
    print(graves[["relación", "%_claves", "%_filas"]].to_string(index=False))

CLAVES = [("appearances", "player_id"), ("appearances", "game_id"),
          ("appearances", "minutes_played"), ("appearances", "competition_id"),
          ("games", "game_id"), ("games", "season"),
          ("player_valuations", "player_id"),
          ("player_valuations", "market_value_in_eur"),
          ("club_games", "club_id"), ("club_games", "is_win"),
          ("clubs", "domestic_competition_id")]

print("\n=== COLUMNAS CRITICAS (ausencias esperadas: cero) ===")
for t, c in CLAVES:
    if t in df and c in df[t].columns:
        n = df[t][c].isna().sum()
        print(f" {t}. {c}:<28 {n:>7,} nulos  {'OK' if n == 0 else '<-- REVISAR'}")

print(f"\nCobertura temporal de games.season: "
      f"{df['games']['season'].min()} - {df['games']['season'].max()}")

```

3.3. Consistencia interna

Se comprueba que los registros no se contradicen entre sí ni consigo mismos.

La ausencia no relevante de incidencias es en sí misma un resultado, ya que garantiza que las agregaciones posteriores no arrastran errores de origen.

incidencias = []

```

def chk(descripcion, n, total):
    incidencias.append({"comprobación": descripcion,
                        "casos": int(n),
                        "%": round(100 * n / max(total, 1), 3)})

# --- 1. Imposibles físicos -----
ap = df["appearances"]
n = len(ap)
chk("minutes_played negativo",      (ap["minutes_played"] < 0).sum(), n)
chk("minutes_played > 120",         (ap["minutes_played"] > 120).sum(), n)
chk("goals negativo",               (ap["goals"] < 0).sum(), n)
chk("yellow_cards > 2 en un partido", (ap["yellow_cards"] > 2).sum(), n)
chk("red_cards > 1 en un partido",   (ap["red_cards"] > 1).sum(), n)

```

```

pl = df["players"]
np_ = len(pl)
nac = pd.to_datetime(pl["date_of_birth"], errors="coerce")
chk("fecha de nacimiento nula o ilegible", nac.isna().sum(), np_)
chk("nacimiento fuera de 1950-2010",
    (~nac.dt.year.between(1950, 2010) & nac.notna()).sum(), np_)
chk("altura fuera de 150-215 cm",
    (~pl["height_in_cm"].between(150, 215) & pl["height_in_cm"].notna()).sum(), np_)

val = df["player_valuations"]
nv = len(val)
chk("valor de mercado <= 0", (val["market_value_in_eur"] <= 0).sum(), nv)

# --- 2. Coherencia temporal partido / temporada -----
g = df["games"].copy()
g["date"] = pd.to_datetime(g["date"])
ini = pd.to_datetime(g["season"].astype(str) + "-07-01")
fin = pd.to_datetime((g["season"] + 1).astype(str) + "-06-30")
chk("partido fuera del rango de su temporada",
    (~g["date"].between(ini, fin)).sum(), len(g))

# --- 3. Duplicados lógicos -----
chk("appearance_id duplicado", ap["appearance_id"].duplicated().sum(), n)
chk("mismo jugador dos veces en el mismo partido",
    ap.duplicated(["game_id", "player_id"]).sum(), n)
chk("misma valoración duplicada (jugador + fecha)",
    val.duplicated(["player_id", "date"]).sum(), nv)

# --- 4. Coherencia entre apariciones y partidos -----
cmp_ = ap[["game_id", "player_club_id"]].merge(
    df["games"][["game_id", "home_club_id", "away_club_id"]],
    on="game_id", how="inner")
incoherente = ~(cmp_["player_club_id"] == cmp_["home_club_id"]) |
    (cmp_["player_club_id"] == cmp_["away_club_id"])
chk("club del jugador ajeno al partido", incoherente.sum(), len(cmp_))

# --- Resultado -----
auditoria = pd.DataFrame(incidencias)
auditoria.to_csv(PROCESSED / "anexoB_consistencia.csv", index=False, encoding="utf-8")

print("=== CONSISTENCIA INTERNA ===")
print(auditoria.to_string(index=False))

criticas = auditoria[auditoria["%"] > 3]
print("\nSin incidencias por encima del 3%: los datos son consistentes."
    if criticas.empty else f"\nATENCIÓN, revisar:\n{criticas.to_string(index=False)}")
## 3.4. Cobertura temporal

```

Esta es la dimensión más determinante del estudio, ya que la variable objetivo se define sobre una ventana temporal concreta. Se analiza cómo se distribuyen las valoraciones a lo

```

largo del tiempo y si la cobertura es homogénea durante todo el periodo.
pv = df["player_valuations"].copy()
COL_FECHA_VAL = next(c for c in ["date", "datetime"] if c in pv.columns)
pv[COL_FECHA_VAL] = pd.to_datetime(pv[COL_FECHA_VAL])

print("Rango temporal:", pv[COL_FECHA_VAL].min().date(), "→",
pv[COL_FECHA_VAL].max().date())

print("\nValoraciones por año:")
print(pv[COL_FECHA_VAL].dt.year.value_counts().sort_index().loc[2010:].to_string())

print("\nDistribución por mes (identificación de oleadas de revisión):")
print(pv[COL_FECHA_VAL].dt.month.value_counts().sort_index().to_string())

print("\nValoraciones por jugador y año:")
print(pv.groupby(["player_id", pv[COL_FECHA_VAL].dt.year]).size()
      .describe()[["count", "mean", "50%", "max"]].to_string())

print("\nJugadores distintos CON VALORACIÓN, por año:")
print(pv.groupby(pv[COL_FECHA_VAL].dt.year)["player_id"].nunique().loc[2015:].to_string())

ap_ = df["appearances"].copy()
ap_["date"] = pd.to_datetime(ap_["date"])
print("\nJugadores distintos CON APARICIONES, por año:")
print(ap_.groupby(ap_["date"].dt.year)["player_id"].nunique().loc[2015:].to_string())
BIG5_DIAG = ["GB1", "ES1", "IT1", "L1", "FR1"]
apps_d = df["appearances"].merge(df["games"][["game_id", "season"]],
                                on="game_id", how="left")
apps_d["date"] = pd.to_datetime(apps_d["date"])

print("\nCobertura de una ventana 1-may / 15-jul de 2024, por liga (temporada 2023/24):")
for lg in BIG5_DIAG:
    jug = set(apps_d.loc[(apps_d.season == 2023) &
                        (apps_d.competition_id == lg), "player_id"])
    v = pv[pv["player_id"].isin(jug) &
           pv[COL_FECHA_VAL].between("2024-06-01", "2024-07-15")]
    print(f" {lg}: {len(jug):>4} jugadores | {v['player_id'].nunique():>4} con valoración "
          f"f'({v['player_id'].nunique()/max(len(jug),1):>6.1%})")

print("\nMes de revisión en 2024, por liga (jugadores distintos):")
for lg in BIG5_DIAG:
    jug = set(apps_d.loc[(apps_d.season == 2023) &
                        (apps_d.competition_id == lg), "player_id"])
    v24 = pv[pv["player_id"].isin(jug) & (pv[COL_FECHA_VAL].dt.year == 2024)]
    serie = v24.groupby(v24[COL_FECHA_VAL].dt.month)["player_id"].nunique()
    print(f" {lg}: " + " | ".join(f"m{m}:{n}" for m, n in serie.items()))

print("\nÚltimo partido de la temporada 2023/24, por liga:")
for lg in BIG5_DIAG:
    s = apps_d[(apps_d.season == 2023) & (apps_d.competition_id == lg)]
    print(f" {lg}: {s['date'].max().date()}")

```

```
print("\nValoraciones en la ventana 1-may / 15-jul, por año:")
for anio in range(2016, 2026):
    v = pv[pv[COL_FECHA_VAL].between(f"{anio}-05-01", f"{anio}-07-15")]
    print(f" {anio}: {len(v):>7,} valoraciones | "
          f"{v['player_id'].nunique():>6,} jugadores distintos")
## 3.5. Conclusiones de la auditoría
```

Los hallazgos y las decisiones que se derivan de ellos:

- **Oleadas de revisión**: las valoraciones no se actualizan de forma continua, sino en dos oleadas anuales, una en primavera y otra en diciembre, coherente con una mediana de dos valoraciones por jugador y año. La oleada de primavera no ocurre en una fecha única: se distribuye a lo largo de múltiples fechas.
- **Las ligas no se revisan simultáneamente**. La Premier League (GB1) y la Bundesliga (L1) concluyen antes que las ligas de España, Francia e Italia, y se revisan en **mayo**, mientras que LaLiga, la Serie A y la Ligue 1 se revisan en **junio**. Para poder abarcar estas 5 grandes ligas, la ventana del objetivo se fija en **1 de mayo – 15 de julio**, tomando la última valoración disponible.
- **Cobertura estable desde 2016**. A partir de 2016 se superan sistemáticamente las 34.000 valoraciones anuales, mientras que los años anteriores son significativamente menores. En consecuencia, el periodo de estudio será a partir de la temporada 2016/2017.
- **Anomalía de 2020**. La ventana de primavera de 2020 registra una fracción de las valoraciones habituales, consecuencia de la interrupción de las competiciones por la pandemia. Por ello, excluye la temporada 2019/2020.
- **Caída de cobertura desde 2024**. Los jugadores distintos con valoración descienden de 27.890 en 2023 a 19.628 en 2024, mientras que los jugadores con apariciones aumentan en ese mismo periodo. En consecuencia, el conjunto de prueba se limita a la temporada 2023/2024 y no se emplea 2024/25.

3.6. Checkpoint

Las tablas se guardan en formato Parquet, que conserva los tipos de datos y acelera notablemente la lectura. A partir de aquí el proceso de construcción parte de estos ficheros, lo que permite iterar sobre la transformación sin repetir la carga de los CSV.

```
for nombre, tabla in df.items():
    tabla.to_parquet(INTERIM / f"{nombre}.parquet", index=False)
    print(f"{nombre:<20} guardado")
print("\nCheckpoint completo en", INTERIM)
---
```

4. Construcción del dataset analítico

El diseño temporal

La decisión de ventana temporal para este proyecto es:

Temporada S (2023 = 2023/24)	CORTE	VENTANA OBJETIVO
[--- ago 2023 ... may 2024 ---]	1 may 2024	[1 may - 15 jul 2024]

<----- PREDICTORES ----->

<----- OBJETIVO ----->

...

- Ninguna variable explicativa puede usar información posterior al 1 de mayo.
- La variable objetivo representa la última valoración de un jugador en la temporada.
- La unidad de análisis es una fila que representa a un jugador en una temporada

4.1. Parámetros del estudio

Todos los parámetros están justificados en las conclusiones de la auditoría (sección 3.5) o en el capítulo correspondiente de la memoria. Centralizarlos en una única celda permite reproducir el estudio con otra configuración modificando un solo punto.

--- Parámetros del estudio -----

SEASONS = list(range(2016, 2024)) # 2016/17 ... 2023/24

SEASON_EXCLUIR = [2019] # Se excluye la temporada 2019/2020 por irregularidades a consecuencia del covid

BIG5 = ["GB1", "ES1", "IT1", "L1", "FR1"] # Se seleccionan para el estudio las 5 grandes ligas europeas

POSICIONES = ["Attack", "Midfield", "Defender"] # Se descartan los porteros por las evidentes diferencias en rendimiento respecto al resto de la plantilla

MIN_MINUTOS = 450 # equivalente a 5 partidos completos

VENTANA_TARGET = (5, 1, 7, 15) # (mes_ini, dia_ini, mes_fin, dia_fin)

MES_CORTE, DIA_CORTE = 5, 1 # fin del periodo de rendimiento

SPLIT = {"train": [2016, 2017, 2018, 2020, 2021],
"valid": [2022],
"test": [2023]}

SEASONS = [s for s in SEASONS if s not in SEASON_EXCLUIR]
print("Temporadas del estudio:", SEASONS)

--- Carga desde los checkpoints -----

tablas = ["appearances", "player_valuations", "players", "clubs",
"competitions", "games", "game_lineups", "club_games"]

df = {}

for t in tablas:

 ruta = INTERIM / f"{t}.parquet"

 if ruta.exists():

 df[t] = pd.read_parquet(ruta)

 print(f"{t}<20} {df[t].shape[0]:>10,} x {df[t].shape[1]:>3}")

 else:

 print(f"AVISO: falta {t}.parquet")

--- Detección defensiva de nombres de columna -----

def col_fecha_de(tabla, candidatas=("date", "datetime", "transfer_date")):

 for c in candidatas:

 if c in df[t].columns:

 return c

 raise KeyError(f"No encuentro columna de fecha en {tabla}")

COL_FECHA_VAL = col_fecha_de("player_valuations")

COL_FECHA_APP = col_fecha_de("appearances")

```
print(f"\nFecha en valuations: '{COL_FECHA_VAL}' | en appearances: '{COL_FECHA_APP}'")
```

```
df["player_valuations"][COL_FECHA_VAL] =  
pd.to_datetime(df["player_valuations"][COL_FECHA_VAL])  
df["appearances"][COL_FECHA_APP] = pd.to_datetime(df["appearances"][COL_FECHA_APP])  
## 4.2. Agregación jugador-temporada
```

`appearances` no incluye la temporada, por lo que se obtiene cruzando con `games`. A continuación se agrupa por jugador y temporada, sumando la producción de todos sus partidos.

Se conserva el desglose por tipo de competición, ya que la exposición a competición europea es un indicador relevante del nivel del jugador.

'appearances' ya incluye competition_id: de 'games' solo se necesita 'season'.

```
apps = df["appearances"].merge(df["games"][["game_id", "season"]],  
                              on="game_id", how="left")  
assert "competition_id" in apps.columns
```

```
if "type" in df["competitions"].columns:
```

```
    apps = apps.merge(  
        df["competitions"][["competition_id", "type"]].rename(columns={"type": "comp_type"}),  
        on="competition_id", how="left")
```

```
else:
```

```
    apps["comp_type"] = np.nan
```

```
print("Tipos de competición encontrados:")
```

```
print(apps["comp_type"].value_counts(dropna=False).to_string())
```

```
apps = apps[apps["season"].isin(SEASONS)].copy()
```

Corte de rendimiento: se excluyen los partidos posteriores al 1 de mayo de
S+1, fecha de apertura de la ventana de valoración. Sin este corte, las
últimas jornadas serían posteriores al target en las ligas que se revisan
antes (Premier y Bundesliga).

```
apps["_corte"] = pd.to_datetime((apps["season"] + 1).astype(str) +  
                                f"-{MES_CORTE:02d}-{DIA_CORTE:02d}")
```

```
antes = len(apps)
```

```
apps = apps[apps[COL_FECHA_APP] < apps["_corte"]].copy()
```

```
print(f"\nApariciones excluidas por posteriores al corte: "
```

```
      f"{antes - len(apps):,} ({(antes - len(apps)) / antes:.1%})")
```

```
print(f"Apariciones en las temporadas del estudio: {len(apps):,}")
```

```
apps["es_liga"] = (apps["comp_type"] == "domestic_league").astype(int)
```

```
apps["es_euro"] = apps["competition_id"].isin(["CL", "EL", "UCOL"]).astype(int)
```

```
perf = (apps.groupby(["player_id", "season"])
```

```
    .agg(partidos      = ("game_id", "nunique"),  
         minutos      = ("minutes_played", "sum"),  
         goles        = ("goals", "sum"),  
         asistencias  = ("assists", "sum"),  
         amarillas    = ("yellow_cards", "sum"),  
         rojas        = ("red_cards", "sum"),
```

```
partidos_liga = ("es_liga", "sum"),
partidos_euro = ("es_euro", "sum"),
n_competiciones = ("competition_id", "nunique"))
.reset_index())
```

```
print(f"Filas jugador-temporada tras agregar: {len(perf):,}")
```

```
assert perf.duplicated(["player_id", "season"]).sum() == 0
```

```
## 4.3. Club de la temporada, forma del club y rol
```

****No se emplea `players.current\_club\_id`**: refleja el club actual del jugador, no el que tenía en la temporada analizada. Para un jugador que cambió de equipo —o que ya se ha retirado— sería sencillamente incorrecto.**

El club de cada temporada se deduce de ****aquel en el que acumuló más minutos****, información que procede directamente de las apariciones y por tanto es históricamente correcta.

Calculo del club en el que jugo durante la temporada (mas minutos jugados)

```
club_temp = (apps.groupby(["player_id", "season", "player_club_id"])["minutes_played"]
              .sum().reset_index()
              .sort_values("minutes_played", ascending=False)
              .drop_duplicates(["player_id", "season"])
              .rename(columns={"player_club_id": "club_id"})
              [["player_id", "season", "club_id"]])
```

```
perf = perf.merge(club_temp, on=["player_id", "season"], how="left")
```

```
perf["liga"] = perf["club_id"].map(df["clubs"].set_index("club_id")["domestic_competition_id"])
```

```
# --- Forma del club en esa temporada -----
```

```
cg = df["club_games"].merge(df["games"][["game_id", "season", "competition_id", "date"]],
                           on="game_id", how="left")
```

```
cg = cg[cg["competition_id"].isin(BIG5)].copy() # solo liga doméstica
```

Mismo corte que en el rendimiento individual: la forma del club no puede

incorporar jornadas posteriores a la fecha de corte.

```
cg["date"] = pd.to_datetime(cg["date"])
```

```
cg = cg[cg["date"] < pd.to_datetime((cg["season"] + 1).astype(str) +
                                     f"-{MES_CORTE:02d}-{DIA_CORTE:02d}")]
```

```
cg["empate"] = (cg["own_goals"] == cg["opponent_goals"]).astype(int)
```

```
cg["puntos"] = cg["is_win"] * 3 + cg["empate"]
```

```
forma = (cg.groupby(["club_id", "season", "competition_id"])
         .agg(cg_partidos=("puntos", "size"), cg_puntos=("puntos", "sum"),
             cg_gf=("own_goals", "sum"), cg_gc=("opponent_goals", "sum"))
         .reset_index())
```

```
forma = forma[forma["cg_partidos"] >= 10] # se descartan aquellos equipos con menos de 10
partidos disputados
```

```
forma["club_ppp"] = forma["cg_puntos"] / forma["cg_partidos"]
```

```
forma["club_dif_g"] = (forma["cg_gf"] - forma["cg_gc"]) / forma["cg_partidos"]
```

```
forma["club_rank"] = forma.groupby(["competition_id", "season"])["cg_puntos"] \
    .rank(ascending=False, method="min")
```

```

perf = perf.merge(forma[["club_id", "season", "club_ppp", "club_dif_g", "club_rank"]],
                  on=["club_id", "season"], how="left")

# Titularidad y capitanes
gl = df["game_lineups"].merge(df["games"][["game_id", "season"]], on="game_id", how="left")
gl = gl[gl["season"].isin(SEASONS)]
gl["es_titular"] = (gl["type"].astype(str).str.lower() == "starting_lineup").astype(int)
gl["es_capitan"] = gl["team_captain"].fillna(0).astype(int) \
                  if "team_captain" in gl.columns else 0

rol = (gl.groupby(["player_id", "season"])
      .agg(convocatorias=("game_id", "nunique"),
          titularidades=("es_titular", "sum"),
          capitancias =("es_capitan", "sum"))
      .reset_index())

perf = perf.merge(rol, on=["player_id", "season"], how="left")
for c in ["convocatorias", "titularidades", "capitancias"]:
    perf[c] = perf[c].fillna(0)

print(f"Tras club, forma y rol: {len(perf):,} filas")
print(f"Sin liga asignada: {perf['liga'].isna().mean():.2%}")
## 4.4. Perfil del jugador y edad a fecha de corte
pl = df["players"].copy()
pl["date_of_birth"] = pd.to_datetime(pl["date_of_birth"], errors="coerce")

cols_perfil = [c for c in ["player_id", "date_of_birth", "position", "sub_position",
                          "foot", "height_in_cm", "country_of_citizenship"]
               if c in pl.columns]
perf = perf.merge(pl[cols_perfil], on="player_id", how="left")

perf["fecha_corte"] = pd.to_datetime((perf["season"] + 1).astype(str) +
                                     f"-{MES_CORTE:02d}-{DIA_CORTE:02d}")
perf["edad"] = (perf["fecha_corte"] - perf["date_of_birth"]).dt.days / 365.25

print(perf.groupby("season")["edad"]
      .describe()[["count", "mean", "min", "max"]].round(1).to_string())
## 4.5. Valoración previa y valor de plantilla

```

Se recupera la última valoración conocida antes del inicio de la temporada.

Es un predictor legítimo: se sitúa unos diez meses antes de la fecha de corte. No obstante, al capturar la persistencia del mercado, se reserva para la **especificación de robustez** y no forma parte del modelo principal.

A partir de esta misma variable se deriva el valor medio de la plantilla del club en esa temporada, que sustituye al valor actual de `clubs` como indicador de dimensión económica.

```

val = (df["player_valuations"][["player_id", COL_FECHA_VAL, "market_value_in_eur"]]
      .rename(columns={COL_FECHA_VAL: "fecha", "market_value_in_eur": "valor"})
      .dropna(subset=["valor"])
      .sort_values("fecha"))

```

```

perf["inicio_temporada"] = pd.to_datetime(perf["season"].astype(str) + "-08-01")

izq = perf[["player_id", "season", "inicio_temporada"]].sort_values("inicio_temporada")
mv_prev = pd.merge_asof(izq, val,
                        left_on="inicio_temporada", right_on="fecha",
                        by="player_id", direction="backward",
                        tolerance=pd.Timedelta(days=400)).rename(columns={"valor": "mv_prev"})

perf = perf.merge(mv_prev[["player_id", "season", "mv_prev", "fecha"]],
                 .rename(columns={"fecha": "mv_prev_fecha"}),
                 on=["player_id", "season"], how="left")

perf["dias_mv_prev"] = (perf["inicio_temporada"] - perf["mv_prev_fecha"]).dt.days
perf["mv_prev_ausente"] = perf["mv_prev"].isna().astype(int)
print(perf["dias_mv_prev"].describe(percentiles=[.5, .9, .99]).round(0).to_string())
print(f"Emparejamientos de mas de un ano: {(perf['dias_mv_prev'] > 365).mean():.2%}")

# Valor medio de plantilla: histórico y coherente con la fecha de corte
plantilla = (perf.dropna(subset=["mv_prev"])
             .groupby(["club_id", "season"])["mv_prev"]
             .agg(club_valor_medio="mean", club_n_valorados="size").reset_index())
perf = perf.merge(plantilla, on=["club_id", "season"], how="left")

# Aforo: foto actual, pero el tamaño del estadio es muy estable en el tiempo.
# Se admite como proxy de dimensión del club y se declara en limitaciones.
if "stadium_seats" in df["clubs"].columns:
    perf["club_aforo"] = perf["club_id"].map(
        df["clubs"].set_index("club_id")["stadium_seats"])

print(f"Filas con valoración previa: {perf['mv_prev'].notna().mean():.1%}")
### 4.6. Variable objetivo

```

Para cada temporada se selecciona la ****última valoración**** de cada jugador dentro de la ventana comprendida entre el 1 de mayo y el 15 de julio del año siguiente. Es la valoración vigente al abrir el mercado de fichajes de verano, una vez concluida la temporada.

La ventana abarca un periodo amplio porque, según se comprobó en la auditoría, la revisión de mayo no se produce en una fecha concreta sino distribuida a lo largo del mes.

```
m_ini, d_ini, m_fin, d_fin = VENTANA_TARGET
```

```

filas_t = []
for s in SEASONS:
    ini = pd.Timestamp(year=s + 1, month=m_ini, day=d_ini)
    fin = pd.Timestamp(year=s + 1, month=m_fin, day=d_fin)
    sub = val[(val["fecha"] >= ini) & (val["fecha"] <= fin)].copy()
    sub = sub.sort_values("fecha").drop_duplicates("player_id", keep="last")
    sub["season"] = s
    filas_t.append(sub[["player_id", "season", "valor", "fecha"]])

```

```

target = (pd.concat(filas_t, ignore_index=True)
          .rename(columns={"valor": "target_valor", "fecha": "target_fecha"}))

print("Objetivos disponibles por temporada:")
print(target.groupby("season").size().to_string())

perf = perf.merge(target, on=["player_id", "season"], how="left")
print(f"\nFilas con objetivo: {perf['target_valor'].notna().sum():,} de {len(perf):,} "
      f"({perf['target_valor'].notna().mean():.1%})")
### 4.7. Ratios, filtros de población y deflactado
d = perf.copy()

# --- Métricas por 90 minutos -----
noventas = d["minutos"] / 90.0
d["goles_90"] = d["goles"] / noventas.replace(0, np.nan)
d["asistencias_90"] = d["asistencias"] / noventas.replace(0, np.nan)
d["ga_90"] = (d["goles"] + d["asistencias"]) / noventas.replace(0, np.nan)
d["amarillas_90"] = d["amarillas"] / noventas.replace(0, np.nan)

# --- Rol relativo -----
d["min_por_partido"] = d["minutos"] / d["partidos"].replace(0, np.nan)
d["ratio_titular"] = d["titularidades"] / d["convocatorias"].replace(0, np.nan)
d["jugo_europa"] = (d["partidos_euro"] > 0).astype(int)

# --- Filtros de población -----
EMBUDO = [("Jugador-temporada inicial", len(d))]

d = d[d["target_valor"].notna()] ; EMBUDO.append(("Con valoración objetivo", len(d)))
d = d[d["liga"].isin(BIG5)] ; EMBUDO.append(("Ligas Big 5", len(d)))
d = d[d["minutos"] >= MIN_MINUTOS] ; EMBUDO.append(("Mínimo {MIN_MINUTOS} minutos", len(d)))
d = d[d["position"].isin(POSICIONES)] ; EMBUDO.append(("Jugadores de campo", len(d)))
d = d[d["edad"].between(15, 45)] ; EMBUDO.append(("Edad plausible", len(d)))

embudo = pd.DataFrame(EMBUDO, columns=["Criterio", "Filas"])
embudo["Retenido_%"] = (100 * embudo["Filas"] / embudo["Filas"].iloc[0]).round(1)
print("=== EMBUDO DE FILTRADO ===")
print(embudo.to_string(index=False))
# # --- Índice de mercado (desfasado, sin informacion futura) -----
COL_COMP = "player_club_domestic_competition_id"
val_idx = (df["player_valuations"]
          .rename(columns={COL_FECHA_VAL: "fecha",
                          "market_value_in_eur": "valor"})
          .dropna(subset=["valor"]))
val_idx["fecha"] = pd.to_datetime(val_idx["fecha"])

if COL_COMP in val_idx.columns:
    val_idx = val_idx[val_idx[COL_COMP].isin(BIG5)]
    print(f"Índice restringido a Big 5: {len(val_idx):,} valoraciones")
else:

```

```

print("AVISO: no existe la columna de competicion; indice sin restringir")

filas_idx = []
for s in SEASONS:
    ini = pd.Timestamp(year=s, month=m_ini, day=d_ini)
    fin = pd.Timestamp(year=s, month=m_fin, day=d_fin)
    v = val_idx[(val_idx["fecha"] >= ini) & (val_idx["fecha"] <= fin)]
    filas_idx.append({"season": s, "indice_mercado": v["valor"].median(),
                     "n_indice": len(v),
                     "ventana": f"{ini:%Y-%m-%d} / {fin:%Y-%m-%d}"})

indice = pd.DataFrame(filas_idx)

UMBRAL_N = 0.5 * indice["n_indice"].median()
malas = indice["n_indice"] < UMBRAL_N
if malas.any():
    print(f"Ventana escasa, indice interpolado: {indice.loc[malas, 'season'].tolist()}")
    indice.loc[malas, "indice_mercado"] = np.nan
    indice["indice_mercado"] = indice["indice_mercado"].interpolate().bfill().ffill()

print("\n=== INDICE DE MERCADO (ventana del ano anterior, Big 5) ===")
print(indice.to_string(index=False))
assert indice["indice_mercado"].notna().all(), "Alguna temporada sin indice"

d = d.merge(indice[["season", "indice_mercado"]], on="season", how="left")

assert (d["target_valor"] > 0).all(), "Valores no positivos en el objetivo"

d["log_indice"] = np.log(d["indice_mercado"])
d["target_defl"] = d["target_valor"] / d["indice_mercado"]

d["y_log"] = np.log(d["target_valor"])
d["y_log_defl"] = d["y_log"] - d["log_indice"]

d["mv_prev_defl"] = d["mv_prev"] / d["indice_mercado"]
d["log_mv_prev"] = np.where(d["mv_prev"].notna(),
                             np.log(d["mv_prev"]) - d["log_indice"], np.nan)

print(f"\nAsimetria nominal: {d['target_valor'].skew():.2f} | "
      f"log nominal: {d['y_log'].skew():.2f} | "
      f"log deflactado: {d['y_log_defl'].skew():.2f}")
print("\nDeriva residual (mediana del objetivo deflactado por temporada):")
print(d.groupby("season")["target_defl"].median().round(3).to_string())
## 4.8. Ensamblado final, partición temporal y guardado

```

Se seleccionan las columnas definitivas y se marca la partición.

La partición es temporal, no aleatoria, ya que un reparto aleatorio situaría temporadas futuras en entrenamiento y pasadas en prueba, evaluando el modelo en condiciones que no se dan en la práctica.

```

FEAT_NUM = [
    "edad", "height_in_cm",                                # perfil
    "partidos", "minutos", "goles", "asistencias", "amarillas", "rojas", # producción
    "goles_90", "asistencias_90", "ga_90", "amarillas_90",    # eficiencia
    "min_por_partido", "ratio_titular", "capitanias",        # rol
    "partidos_liga", "partidos_euro", "n_competiciones", "jugo_europa", # competición
    "club_ppp", "club_dif_g", "club_rank", "club_valor_medio", "club_aforo", # club
]
FEAT_CAT = ["position", "sub_position", "foot", "liga"]
FEAT_MV = ["log_mv_prev", "mv_prev_ausente"]

FEAT_NUM = [c for c in FEAT_NUM if c in d.columns]
FEAT_CAT = [c for c in FEAT_CAT if c in d.columns]

ID_COLS = ["player_id", "season", "club_id", "target_fecha",
            "indice_mercado", "dias_mv_prev"]
Y_COLS = ["target_valor", "target_defl", "y_log", "y_log_defl"]

analitico = d[ID_COLS + FEAT_NUM + FEAT_CAT + FEAT_MV + Y_COLS].copy()

analitico["split"] = np.select(
    [analitico["season"].isin(SPLIT["train"]),
     analitico["season"].isin(SPLIT["valid"]),
     analitico["season"].isin(SPLIT["test"])],
    ["train", "valid", "test"], default="fuera")

analitico.to_parquet(PROCESSED / "dataset_analitico.parquet", index=False)
print("Guardado en", PROCESSED / "dataset_analitico.parquet")
print(f"Dimensión: {analitico.shape[0]:,} filas x {analitico.shape[1]} columnas")
print("=== FILAS POR TEMPORADA Y LIGA ===")
print(pd.crosstab(analitico["season"], analitico["liga"], margins=True).to_string())

print("\n=== EMBUDO DE FILTRADO ===")
print(embudo.to_string(index=False))

print("\n=== DIMENSIÓN Y PARTICIÓN ===")
print(f"Filas: {analitico.shape[0]:,} | Jugadores: {analitico['player_id'].nunique():,}")
print(analitico.groupby("split").size().reindex(["train", "valid", "test"]).to_string())

print("\n=== COBERTURA DE TARGET POR LIGA (temporada 2023) ===")
p23 = perf[perf.season == 2023]
print(p23.groupby("liga")["target_valor"].agg(
    n="size", con_target=lambda s: s.notna().sum(),
    pct=lambda s: f"{s.notna().mean():.1%}").to_string())
---
```

5. Verificación de la salida

Antes de dar por buena la transformación se comprueba que el conjunto generado cumple las propiedades que se le presuponen. Estas verificaciones son la garantía de que los modelos del notebook 2 se entrenan sobre datos correctos.

5.1. Comprobaciones de integridad

```

a = pd.read_parquet(PROCESSED / "dataset_analitico.parquet")

pruebas = []
def verifica(descripcion, condicion, detalle=""):
    pruebas.append({"comprobación": descripcion,
                    "resultado": "OK" if condicion else "FALLO",
                    "detalle": detalle})

verifica("La clave jugador-temporada es única",
        a.duplicated(["player_id", "season"]).sum() == 0,
        f"{a.duplicated(['player_id', 'season']).sum()} duplicados")
verifica("El objetivo no tiene valores ausentes",
        a["y_log_defl"].notna().all())
verifica("Ninguna temporada aparece en dos particiones",
        not (set(a.loc[a.split=="train", "season"]) & set(a.loc[a.split=="test", "season"])))
verifica("Todas las filas tienen partición asignada",
        (a["split"] != "fuera").all())
verifica("Edad dentro de un rango plausible",
        a["edad"].between(15, 45).all(),
        f"[{a['edad'].min():.1f}, {a['edad'].max():.1f}]")
verifica("Minutos por encima del umbral fijado",
        (a["minutos"] >= MIN_MINUTOS).all())
verifica("Ratio de titularidad entre 0 y 1",
        a["ratio_titular"].between(0, 1).all())
verifica("Sin ratios por 90 imposibles (goles_90 < 3)",
        (a["goles_90"] < 3).all(), f"máx {a['goles_90'].max():.2f}")
verifica("El objetivo es estrictamente positivo",
        (a["target_valor"] > 0).all())
verifica("Sin porteros en la muestra",
        "Goalkeeper" not in a["position"].unique())

resultado = pd.DataFrame(pruebas)
print(resultado.to_string(index=False))
print("\nTODAS LAS COMPROBACIONES SUPERADAS"
      if (resultado["resultado"] == "OK").all() else "\nHAY FALLOS: revisar")

print("\n--- Nulos por columna (solo las que tienen) ---")
nulos = a.isna().mean()
print((nulos[nulos > 0] * 100).round(2).to_string() if (nulos > 0).any() else " ninguno")
## 5.2. Resumen del conjunto generado

Tablas formateadas para su incorporación directa a la memoria.
print("=== DIMENSIÓN ===")
print(f"Filas: {a.shape[0]:,}")
print(f"Columnas: {a.shape[1]}")
print(f"Jugadores distintos: {a['player_id'].nunique():,}")
print(f"Temporadas por jugador (media): {a.shape[0]/a['player_id'].nunique():.2f}")

print("\n=== EMBUDO DE FILTRADO (memoria, cap. 4) ===")
print(embudo.to_string(index=False))

```

```

print("\n=== REPARTO POR PARTICIÓN ===")
rep = (a.groupby("split").agg(filas=("player_id", "size"),
                             jugadores=("player_id", "nunique"),
                             temporadas=("season", "nunique"))
      .reindex(["train", "valid", "test"]))
print(rep.to_string())

print("\n=== FILAS POR TEMPORADA ===")
print(a.groupby(["season", "split"]).size().unstack(fill_value=0).to_string())

print("\n=== DISTRIBUCIÓN POR LIGA Y POSICIÓN ===")
print(pd.crosstab(a["liga"], a["position"], margins=True).to_string())

```

5.3. Trazabilidad: de los datos en crudo a la fila final

Se sigue el recorrido de un jugador concreto desde sus apariciones individuales hasta la fila que lo representa en el conjunto analítico. La comprobación demuestra que la agregación es correcta y permite validar manualmente el cálculo de las variables derivadas.

```

# Un jugador con muchos minutos en una temporada cualquiera
pid = a.query("season == 2022 and minutos > 2500")["player_id"].iloc[0]
nombre = df["players"].query("player_id == @pid")["name"].iloc[0]

print(f"=== {nombre} (player_id {pid}), temporada 2022/23 ===")

crudo = apps.query("player_id == @pid and season == 2022")
print(f"\n--- EN CRUDO: {len(crudo)} apariciones (se muestran 5) ---")
print(crudo[["date", "competition_id", "minutes_played", "goals", "assists"]]
      .head().to_string(index=False))

print(f"\nSumas manuales -> minutos: {crudo['minutes_played'].sum()}, "
      f"goles: {crudo['goals'].sum()}, asistencias: {crudo['assists'].sum()}")

fila = a.query("player_id == @pid and season == 2022")
print("\n--- FILA RESULTANTE EN EL CONJUNTO ANALÍTICO ---")
print(fila.T.to_string())

# Verificación aritmética de una variable derivada
g90 = fila["goles"].iloc[0] / (fila["minutos"].iloc[0] / 90)
print(f"\nComprobación: goles_90 calculado = {g90:.3f} | "
      f"almacenado = {fila['goles_90'].iloc[0]:.3f}")

```

Cierre

El conjunto analítico está construido y verificado en
`data/processed/dataset\_analitico.parquet`.

****Ficheros generados:****

Fichero	Contenido
`dataset_analitico.parquet`	Conjunto de modelado, una fila por jugador y temporada
`anexoA_decisiones_columnas.csv`	Decisión y motivo para cada columna del origen

| `anexoB\_consistencia.csv` | Resultados de las comprobaciones de consistencia |

****Continúa en `02\_modelos.ipynb`**, que parte de este conjunto para el análisis exploratorio y el entrenamiento de los modelos.**

```
print("=== PERFIL DEL CONJUNTO ANALÍTICO ===")
print(a[["edad","minutos","partidos","goles","asistencias","ga_90",
        "ratio_titular","club_ppp","club_rank","club_valor_medio",
        "log_mv_prev","target_valor","y_log_defl"]]
      .describe(percentiles=[.01,.25,.5,.75,.99]).T
      .round(2).to_string())

print("\n=== CATEGÓRICAS ===")
for c in ["position","sub_position","foot","liga"]:
    print(f"\n{c}: {a[c].nunique()} valores")
    print(a[c].value_counts(dropna=False).head(8).to_string())

print("\n=== COBERTURA DE log_mv_prev POR PARTICIÓN ===")
print(a.groupby("split")["log_mv_prev"].apply(lambda s: f"{s.notna().mean():.1%}").to_string())
# =====
# Catálogo de variables del conjunto analítico
# Memoria: cap. 4.3 | Anexo D
# =====

CATALOGO = [
    # (variable, bloque, descripción, origen y cálculo)
    ("edad",          "Perfil", "Edad del jugador a la fecha de corte",
     "(fecha_corte - date_of_birth) / 365,25, con fecha_corte = 1 de mayo de S+1"),
    ("height_in_cm",  "Perfil", "Altura en centímetros", "players.height_in_cm"),
    ("position",      "Perfil", "Posición general (ataque, medio, defensa)", "players.position"),
    ("sub_position",  "Perfil", "Posición específica", "players.sub_position"),
    ("foot",          "Perfil", "Pie dominante", "players.foot; nulos como categoría propia"),

    ("partidos",      "Producción", "Partidos disputados en la temporada",
     "Recuento de game_id distintos en appearances"),
    ("minutos",       "Producción", "Minutos totales jugados",
     "Suma de minutes_played"),
    ("goles",         "Producción", "Goles marcados", "Suma de goals"),
    ("asistencias",   "Producción", "Asistencias", "Suma de assists"),
    ("amarillas",     "Producción", "Tarjetas amarillas", "Suma de yellow_cards"),
    ("rojas",         "Producción", "Expulsiones", "Suma de red_cards"),

    ("goles_90",      "Eficiencia", "Goles por cada 90 minutos jugados",
     "goles / (minutos / 90)"),
    ("asistencias_90", "Eficiencia", "Asistencias por 90 minutos",
     "asistencias / (minutos / 90)"),
    ("ga_90",         "Eficiencia", "Contribución de gol por 90 minutos",
     "(goles + asistencias) / (minutos / 90)"),
    ("amarillas_90",  "Eficiencia", "Amarillas por 90 minutos",
     "amarillas / (minutos / 90)"),

    ("min_por_partido", "Rol", "Minutos medios por partido disputado",
```

```

"minutos / partidos"),
("ratio_titular", "Rol", "Proporción de convocatorias como titular",
"titularidades / convocatorias, desde game_lineups"),
("capitanias", "Rol", "Partidos disputados como capitán",
"Recuento de team_captain en game_lineups"),

("partidos_liga", "Competición", "Partidos en competición doméstica",
"Recuento de apariciones con type = domestic_league"),
("partidos_euro", "Competición", "Partidos en competición europea",
"Recuento de apariciones en CL, EL o UCOL"),
("n_competiciones", "Competición", "Competiciones distintas disputadas",
"Recuento de competition_id distintos"),
("jugo_europa", "Competición", "Indicador de participación europea",
"1 si partidos_euro > 0"),

("club_ppp", "Club", "Puntos por partido del club en su liga",
"(victorias×3 + empates) / partidos, desde club_games"),
("club_dif_g", "Club", "Diferencia de goles por partido del club",
"(goles a favor – en contra) / partidos, desde club_games"),
("club_rank", "Club", "Posición del club en su liga",
"Ranking por puntos dentro de competición y temporada"),
("club_valor_medio", "Club", "Valor medio de la plantilla del club",
"Media de mv_prev de los jugadores del club en esa temporada"),
("club_aforo", "Club", "Capacidad del estadio",
"clubs.stadium_seats; foto actual, admitida por su estabilidad"),

("liga", "Club", "Competición doméstica del jugador",
"Liga en la que acumuló más minutos"),

("log_mv_prev", "Historial", "Valoración previa, deflactada y en logaritmo",
"ln(mv_prev) – ln(indice); solo en la especificación de robustez"),
]

```

```

catalogo = pd.DataFrame(CATALOGO,
                        columns=["variable", "bloque", "descripción", "origen y cálculo"])

```

```

# Se contrasta con las columnas realmente presentes en el conjunto generado
catalogo["en_dataset"] = catalogo["variable"].isin(a.columns)
faltan = set(FEAT_NUM + FEAT_CAT + FEAT_MV) - set(catalogo["variable"])

```

```

catalogo.to_csv(PROCESSED / "anexoC_catalogo_variables.csv",
               index=False, encoding="utf-8")

```

```

print(f"=== CATÁLOGO DE VARIABLES ({len(catalogo)} entradas) ===")
print(catalogo.groupby("bloque").size().to_string())
print(f"\nVariables del modelo sin documentar: {faltan if faltan else 'ninguna'}")
print(f"Entradas no presentes en el conjunto: "
      f"{catalogo.loc[~catalogo['en_dataset'], 'variable'].tolist() or 'ninguna'}")

```

```

catalogo.drop(columns="en_dataset")

```

ANEXO E: CÓDIGO DE NOTEBOOK 02_MODELOS.IPYNB

Estimación del valor de mercado de futbolistas profesionales

Notebook 2 — Análisis exploratorio y modelización

Trabajo Fin de Máster · Máster en Big Data, Data Science e Inteligencia Artificial · Universidad Complutense de Madrid

Qué hace este notebook

```
|||
|---|---|
| Consume | `data/processed/dataset_analitico.parquet` (salida del notebook 1) |
| Produce | Modelos entrenados, tabla comparativa de métricas y el pipeline final serializado |
| Alimenta | Capítulos 3 (Análisis exploratorio) y 5 (Modelización) de la memoria |
| Siguiente | `03_interpretabilidad.ipynb` |
```

Estructura

1. **Análisis exploratorio** — sobre la unidad de análisis real: el jugador-temporada
2. **Especificaciones del modelo** — tres conjuntos de variables con propósitos distintos
3. **Preprocesado** — un tratamiento por familia de algoritmo, siempre dentro del pipeline
4. **Estrategia de validación** — partición temporal, agrupación por jugador y métricas
5. **Modelos** — cinco técnicas en progresión de complejidad
6. **Resultados** — comparativa, análisis de error y selección del modelo final

Nota metodológica

La ingeniería de características se realizó en el notebook 1, ya que las variables derivadas requieren las tablas en crudo. Lo que aquí se aplica es **preprocesado** —imputación, escalado y codificación— que va necesariamente dentro del pipeline para que se ajuste únicamente con los datos de entrenamiento.

No se realiza selección de variables previa. Seleccionar variables observando el rendimiento en validación y reportar después sobre prueba contaminaría la evaluación. Se emplean tres conjuntos fijos definidos por criterio metodológico, y se deja que cada algoritmo gestione la irrelevancia con sus propios mecanismos.

0. Configuración

```
from pathlib import Path
import random, warnings, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
warnings.filterwarnings("ignore")

SEED = 42
random.seed(SEED); np.random.seed(SEED)

PROJ = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
PROCESSED = PROJ / "data" / "processed"
MODELS = PROJ / "models"
FIGURES = PROJ / "reports" / "figures"
for p in (MODELS, FIGURES): p.mkdir(parents=True, exist_ok=True)

sns.set_theme(style="whitegrid")
plt.rcParams.update({"figure.dpi": 110, "font.size": 9})
pd.set_option("display.max_columns", 60)
pd.set_option("display.width", 150)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

a = pd.read_parquet(PROCESSED / "dataset_analitico.parquet")
print(f"{a.shape[0]:,} filas x {a.shape[1]} columnas | "
      f"{a['player_id'].nunique():,} jugadores distintos")
print(a.groupby("split").size().reindex(["train", "valid", "test"]).to_string())
%load_ext watermark
%watermark -v -m -p pandas,numpy,scikit-learn,statsmodels,catboost,shap
---
```

1. Análisis exploratorio

La exploración se realiza sobre el **dataset analítico**, no sobre las tablas en crudo. La razón es que la unidad de análisis del estudio es el jugador-temporada, y esa unidad no existe en los datos de origen: describir `appearances` sería caracterizar partidos individuales, un objeto que el modelo nunca llega a ver.

Cada figura de esta sección justifica una decisión posterior.

1.1. Descriptivos generales

```
NUM_DESC = ["edad", "minutos", "partidos", "goles", "asistencias", "ga_90",
            "ratio_titular", "club_ppp", "club_rank", "club_valor_medio",
            "log_mv_prev", "target_valor"]

desc = a[NUM_DESC].describe(percentiles=[.01, .25, .5, .75, .99]).T
print("=== VARIABLES NUMÉRICAS ===")
print(desc[["count", "mean", "std", "min", "1%", "50%", "99%", "max"]].round(2).to_string())

print("\n=== VARIABLES CATEGÓRICAS ===")
for c in ["position", "sub_position", "foot", "liga"]:
    print(f"\n{c} ({a[c].nunique()} valores):")
    print(a[c].value_counts(dropna=False).to_string())

print("\n=== EL OBJETIVO EN EUROS ===")
q = a["target_valor"].quantile([.01, .25, .5, .75, .99])
print(q.apply(lambda x: f"{x:,.0f} €").to_string())
print(f"\nRatio percentil 99 / percentil 1: {q.loc[.99]/q.loc[.01]:.0f}x")
```

1.2. Figura 1 — Distribución del objetivo

El valor de mercado presenta una asimetría extrema. Sin transformar, cualquier modelo no rendirá correctamente.

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
```

```
ax[0].hist(a["target_valor"] / 1e6, bins=60, color="#4C72B0", edgecolor="white", lw=.3)
ax[0].set_xlabel("Valor de mercado (M€)"); ax[0].set_ylabel("Frecuencia")
ax[0].set_title(f"Escala original (asimetría = {a['target_valor'].skew():.2f})")
```

```
ax[1].hist(a["y_log_defl"], bins=60, color="#55A868", edgecolor="white", lw=.3)
ax[1].set_xlabel("log(valor deflactado)"); ax[1].set_ylabel("")
ax[1].set_title(f"Escala logarítmica (asimetría = {a['y_log_defl'].skew():.2f})")
```

```
fig.suptitle("Distribución del valor de mercado objetivo", y=1.02)
fig.tight_layout(); fig.savefig(FIGURES / "fig1_distribucion_target.png", bbox_inches="tight")
plt.show()
```

```
print(f"Asimetría original : {a['target_valor'].skew():>6.2f}")
print(f"Asimetría log nominal : {a['y_log'].skew():>6.2f}")
print(f"Asimetría log deflact.: {a['y_log_defl'].skew():>6.2f} <- objetivo modelado")
```

1.3. Figura 2 — La curva de valor por edad

La relación entre edad y valor no es monótona. Presenta una fase de revalorización durante los primeros años de carrera, un máximo situado en el entorno de los veinticuatro a veintisiete años, y un declive posterior pronunciado.

Esta forma es determinante para la elección de modelos: una regresión lineal sin transformar asumiría erróneamente que la edad penaliza o premia el valor de manera constante, lo que es falso. De hecho, la correlación lineal entre edad y valor logarítmico es reducida pese a tratarse de uno de los factores más influyentes, precisamente porque la relación es no monótona.

De aquí se derivan dos decisiones de modelado: la incorporación de “splines cúbicos naturales” sobre la edad en el modelo lineal, y el recurso a modelos basados en árboles, capaces de capturar la forma sin especificarla previamente.

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
```

```
for pos, g in a.groupby("position"):
    c = (g.assign(e=g["edad"].round()).groupby("e")["target_valor"].median() / 1e6)
    c = c[(c.index >= 17) & (c.index <= 38)]
    ax[0].plot(c.index, c.values, marker="o", ms=3, lw=1.4, label=pos)
```

```
ax[0].set_xlabel("Edad a fecha de corte"); ax[0].set_ylabel("Valor mediano (M€)")
ax[0].set_title("Por posición"); ax[0].legend(fontsize=8)
```

```
glob = a.assign(e=a["edad"].round()).groupby("e")["target_valor"]
med, n = glob.median()/1e6, glob.size()
```

```
# Se exige un mínimo de observaciones por edad: los extremos del rango
# contienen muy pocos jugadores y su mediana no es representativa.
MIN_N = 50
valida = med[(med.index >= 18) & (med.index <= 37) & (n >= MIN_N)]
```

```

edad_pico = int(valida.idxmax())

ax[1].plot(med.index[(med.index>=17)&(med.index<=38)],
           med.values[(med.index>=17)&(med.index<=38)],
           color="#C44E52", lw=2, marker="o", ms=4, label="Mediana")
ax[1].plot(valida.index, valida.values, color="#C44E52", lw=3, marker="o", ms=5,
           label=f"n ≥ {MIN_N}")
ax[1].axvline(edad_pico, ls="--", c="grey", lw=1)
ax[1].set_xlabel("Edad"); ax[1].set_ylabel("Valor mediano (M€)")
ax[1].set_title(f"Global (máximo a los {edad_pico} años)")
ax[1].legend(fontsize=7)

fig.suptitle("Curva de valor por edad", y=1.02)
fig.tight_layout(); fig.savefig(FIGURES / "fig2_curva_edad.png", bbox_inches="tight")
plt.show()

print(f"Edad de valor máximo (con n >= {MIN_N}): {edad_pico} años")
print(f"Correlación lineal edad-valor (log): "
      f"{np.corrcoef(a['edad'], a['y_log_defl'])[0,1]:.3f} <- baja pese a ser determinante")

print(f"\nObservaciones por edad (extremos del rango):")
print(pd.concat([n.head(6), n.tail(6)]).to_string())
## 1.4. Figura 3 — Persistencia de la valoración

La valoración de un jugador en un año explica en gran medida la del año siguiente.
sub = a.dropna(subset=["log_mv_prev"])
r = np.corrcoef(sub["log_mv_prev"], sub["y_log_defl"])[0, 1]

fig, ax = plt.subplots(1, 2, figsize=(11, 4))

ax[0].scatter(sub["log_mv_prev"], sub["y_log_defl"], s=4, alpha=.15, color="#C44E52")
lim = [sub["log_mv_prev"].min(), sub["log_mv_prev"].max()]
ax[0].plot(lim, lim, "k--", lw=1, label="y = x")
ax[0].set_xlabel("log(valor previo deflactado)"); ax[0].set_ylabel("log(valor objetivo)")
ax[0].set_title(f"Persistencia (r = {r:.3f}, R² = {r**2:.3f})"); ax[0].legend(fontsize=8)

temp = a.groupby("season")["target_valor"].agg(["median", "mean"])
ax[1].plot(temp.index, temp["median"]/1e6, marker="o", label="Mediana")
ax[1].plot(temp.index, temp["mean"]/1e6, marker="s", label="Media")
ax[1].set_xlabel("Temporada"); ax[1].set_ylabel("Valor (M€)")
ax[1].set_title("Nivel de mercado por temporada"); ax[1].legend(fontsize=8)

fig.tight_layout(); fig.savefig(FIGURES / "fig3_persistencia.png", bbox_inches="tight")
plt.show()

print(f"R² de persistencia: {r**2:.3f}")
print("\nEste es el listón real que deben superar los modelos.")
## 1.5. Figura 4 — Correlaciones entre predictores

```

Varios predictores miden aspectos muy próximos —minutos, partidos y titularidades reflejan esencialmente la misma realidad—. Esta redundancia no impide el ajuste de los modelos basados

en árboles, pero ****afecta a la interpretación****: los valores SHAP repartirán el crédito entre variables correlacionadas, de modo que ninguna aparecerá como determinante por sí sola.

```
NUM_CORR = ["edad", "height_in_cm", "partidos", "minutos", "goles", "asistencias",
            "amarillas", "goles_90", "asistencias_90", "ga_90", "min_por_partido",
            "ratio_titular", "capitanias", "partidos_liga", "partidos_euro",
            "club_ppp", "club_dif_g", "club_rank", "club_valor_medio", "club_aforo",
            "y_log_defl"]
NUM_CORR = [c for c in NUM_CORR if c in a.columns]

corr = a[NUM_CORR].corr(method="spearman")

fig, ax = plt.subplots(figsize=(9, 7.5))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, cmap="RdBu_r", center=0, vmin=-1, vmax=1,
            square=True, linewidths=.4, cbar_kws={"shrink": .7}, ax=ax)
ax.set_title("Correlación de Spearman entre predictores y objetivo")
fig.tight_layout(); fig.savefig(FIGURES / "fig4_correlaciones.png", bbox_inches="tight")
plt.show()

print("=== PARES MUY CORRELACIONADOS (|rho| > 0.80) ===")
c2 = corr.drop(index="y_log_defl", columns="y_log_defl").abs()
pares = (c2.where(np.triu(np.ones(c2.shape), k=1).astype(bool))
        .stack().sort_values(ascending=False))
print(pares[pares > .80].round(3).to_string())

print("\n=== CORRELACIÓN CON EL OBJETIVO (top 10) ===")
print(corr["y_log_defl"].drop("y_log_defl").abs()
      .sort_values(ascending=False).head(10).round(3).to_string())
## 1.6. Figura 5 — El efecto del contexto de club
```

Un mismo rendimiento no se valora igual en cualquier equipo. Sin variables de contexto, el modelo atribuiría al rendimiento individual lo que en realidad corresponde a la dimensión del club en el que se juega.

```
fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))

a["_tramo_club"] = pd.qcut(a["club_valor_medio"], 5,
                          labels=["Muy bajo", "Bajo", "Medio", "Alto", "Muy alto"])
sns.boxplot(data=a, x="_tramo_club", y="y_log_defl", ax=ax[0],
            showfliers=False, palette="Blues")
ax[0].set_xlabel("Valor medio de plantilla (quintiles)"); ax[0].set_ylabel("log(valor)")
ax[0].set_title("Dimensión del club")

sns.boxplot(data=a, x="liga", y="y_log_defl", ax=ax[1], showfliers=False, palette="Greens")
ax[1].set_xlabel("Liga"); ax[1].set_ylabel("")
ax[1].set_title("Competición")

rank = a.assign(r=a["club_rank"].round()).groupby("r")["target_valor"].median()/1e6
ax[2].plot(rank.index, rank.values, marker="o", ms=4, color="#C44E52")
ax[2].set_xlabel("Posición del club en liga"); ax[2].set_ylabel("Valor mediano (M€)")
ax[2].set_title("Clasificación del club")
```

```
fig.suptitle("Efecto del contexto de club sobre el valor", y=1.03)
fig.tight_layout(); fig.savefig(FIGURES / "fig5_contexto_club.png", bbox_inches="tight")
plt.show()
```

```
a.drop(columns="_tramo_club", inplace=True)
## 1.7. Valores ausentes y atípicos
```

En este problema los valores extremos son precisamente la parte más relevante. Un jugador de 200 M€ es un dato atípico en todas las variables, pero no es un error: recortarlo haría que el modelo dejase de saber valorar a los mejores jugadores, que es donde se toman las decisiones económicamente más importantes.

La transformación logarítmica ya comprime la cola sin destruir información, y los modelos basados en árboles son insensibles a los valores extremos de los predictores.

```
print("=== VALORES AUSENTES ===")
nul = a.isna().mean()
nul = (nul[nul > 0] * 100).round(2)
print(nul.to_string() if len(nul) else " ninguno")
```

```
print("\n=== TRATAMIENTO PREVISTO ===")
print(" foot      -> categoría propia 'desconocido' (no se eliminan filas)")
print(" club_ppp   -> imputación por mediana dentro del pipeline")
print(" club_rank   -> imputación por mediana dentro del pipeline")
print(" log_mv_prev -> solo afecta a la especificación de robustez")
```

```
print("\n=== CASOS EXTREMOS EN RATIOS POR 90 MINUTOS ===")
print(a.nlargest(5, "ga_90")[["season", "minutos", "partidos", "goles",
                             "asistencias", "ga_90", "target_valor"]].to_string(index=False))
print("\nSon jugadores con pocos minutos y alta participación: efecto de denominador")
print("pequeño, mitigado por el filtro de 450 minutos. No se corrigen.")
```

```
print("\n=== RANGOS DE LAS VARIABLES DERIVADAS ===")
for c in ["goles_90", "asistencias_90", "ga_90", "ratio_titular", "min_por_partido"]:
    print(f" {c:<18} [{a[c].min():.3f}, {a[c].max():.3f}]")
```

```
---
```

2. Especificaciones del modelo

Se definen tres conjuntos de variables. No responden a un proceso de selección automática, sino a una decisión metodológica: cada uno responde a una pregunta distinta.

Especificación	Variables	Pregunta que responde
--- --- ---		
Persistencia	Solo la valoración previa	¿Cuánto se explica sin mirar el rendimiento?
Principal	Rendimiento, perfil y contexto de club	¿Qué predice el rendimiento por sí solo?
Robustez	Todas las anteriores	¿Cuál es el techo alcanzable?

La especificación principal es la del trabajo. Excluye deliberadamente la valoración previa porque, al capturar la persistencia del mercado, ocultaría la relación entre rendimiento y valor, que es el objeto del estudio.

La especificación de robustez se reporta para cuantificar cuánta información adicional aporta

el historial de valoraciones, y sirve de referencia superior.

```
FEAT_NUM = ["edad", "height_in_cm",
            "partidos", "minutos", "goles", "asistencias", "amarillas", "rojas",
            "goles_90", "asistencias_90", "ga_90", "amarillas_90",
            "min_por_partido", "ratio_titular", "capitanias",
            "partidos_liga", "partidos_euro", "n_competiciones", "jugo_europa",
            "club_ppp", "club_dif_g", "club_rank", "club_valor_medio", "club_aforo"]
FEAT_CAT = ["position", "sub_position", "foot", "liga"]
FEAT_MV = ["log_mv_prev"]
```

```
FEAT_NUM = [c for c in FEAT_NUM if c in a.columns]
FEAT_CAT = [c for c in FEAT_CAT if c in a.columns]
```

```
ESPECIFICACIONES = {
    "persistencia": {"num": FEAT_MV, "cat": []},
    "principal": {"num": FEAT_NUM, "cat": FEAT_CAT},
    "robustez": {"num": FEAT_NUM + FEAT_MV, "cat": FEAT_CAT},
}
```

```
for nombre, esp in ESPECIFICACIONES.items():
    print(f"{nombre:<14} {len(esp['num']):>2} numéricas + {len(esp['cat'])} categóricas")
```

```
TARGET = "y_log_defl"
GRUPO = "player_id"
## 2.1. Preparación de los conjuntos
```

Las categóricas ausentes se convierten en una categoría explícita "desconocido" en lugar de eliminar las filas correspondientes. Perder observaciones por un dato de perfil menor sería desproporcionado.

```
d = a.copy()
for c in FEAT_CAT:
    d[c] = d[c].fillna("desconocido").astype(str)
```

```
# La especificación de robustez requiere valoración previa
print(f"Filas sin log_mv_prev: {d['log_mv_prev'].isna().sum()} "
      f"({d['log_mv_prev'].isna().mean():.2%})")
```

```
tr = d[d.split == "train"].reset_index(drop=True)
va = d[d.split == "valid"].reset_index(drop=True)
te = d[d.split == "test"].reset_index(drop=True)
```

```
print(f"\ntrain {len(tr):>6} filas | {tr[GRUPO].nunique():>5} jugadores | "
      f"temporadas {sorted(tr.season.unique())}")
print(f"valid {len(va):>6} filas | {va[GRUPO].nunique():>5} jugadores | "
      f"temporadas {sorted(va.season.unique())}")
print(f"test {len(te):>6} filas | {te[GRUPO].nunique():>5} jugadores | "
      f"temporadas {sorted(te.season.unique())}")
```

```
comunes = set(tr[GRUPO]) & set(te[GRUPO])
print(f"\nJugadores presentes en train y test: {len(comunes):,}")
print("Es esperable y correcto: la separación es temporal, no por jugador.")
```

```
print("Por ello la validación cruzada dentro de train agrupa por player_id.")
```

```
---
```

3. Estrategia de validación y métricas

3.1. Por qué la partición no puede ser aleatoria

Existen dos motivos independientes, y ambos se controlan:

****Motivo temporal.**** Un reparto aleatorio situaría temporadas futuras en entrenamiento y pasadas en prueba. El modelo dispondría de información sobre la evolución del mercado que en un uso real no tendría. La partición es por temporadas completas.

****Motivo de agrupación.**** El conjunto contiene medidas repetidas: 4.372 jugadores generan 11.048 filas, unas 2,5 temporadas por jugador. Un reparto aleatorio situaría al mismo jugador en entrenamiento y en prueba, y el modelo memorizaría individuos en lugar de aprender la relación entre rendimiento y valor. Por ello la validación cruzada dentro de entrenamiento emplea `GroupKFold` agrupando por `player\_id`.

3.2. Métricas

No se emplea el coeficiente de determinación como métrica única. Se reportan:

Métrica	Qué mide
`MAE_log`, `RMSE_log`	Error en la escala en la que se optimiza
`R2_log`	Proporción de varianza explicada
`MAE_eur`, `MedAE_eur`	Error retrotransformado a euros, interpretable
`MAPE_%`	Error relativo medio
`Spearman`	Calidad del **ordenamiento** , relevante para el ranking del notebook 3

****Sobre la retrotransformación.**** Al deshacer el logaritmo mediante la función exponencial se obtiene una estimación de la mediana condicionada, no de la media, lo que produce una subestimación sistemática. Se aplica la corrección de Duan (**smearing estimator**), que ajusta el resultado mediante el promedio de las exponenciales de los residuos de entrenamiento.

```
from sklearn.model_selection import GroupKFold, RandomizedSearchCV, train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from scipy.stats import spearmanr
```

```
CV = GroupKFold(n_splits=5)
```

```
def a_euros(y_log, indice, factor_duan=1.0):
    #  $y = \ln(V) - \ln(I) \Rightarrow V = \exp(y) * I$ , con corrección de Duan.
    # Se fuerza numpy para evitar alineación por índice de pandas.
    return np.exp(np.asarray(y_log)) * factor_duan * np.asarray(indice)
```

```
def duan(residuos_train):
    # Smearing estimator: corrige el sesgo de la retrotransformación
    return float(np.mean(np.exp(residuos_train)))
```

```
def evalua(nombre, especificacion, conjunto, y_true_log, y_pred_log,
            indice, y_true_eur, factor=1.0):
```

```

pred_eur = a_euros(y_pred_log, indice, factor)
err_rel = np.abs(pred_eur - y_true_eur) / y_true_eur
return {
    "modelo": nombre, "espec": especificacion, "conjunto": conjunto,
    "MAE_log": mean_absolute_error(y_true_log, y_pred_log),
    "RMSE_log": np.sqrt(mean_squared_error(y_true_log, y_pred_log)),
    "R2_log": r2_score(y_true_log, y_pred_log),
    "MAE_eur": np.mean(np.abs(pred_eur - y_true_eur)),
    "MedAE_eur": np.median(np.abs(pred_eur - y_true_eur)),
    "MAPE_%": 100 * np.mean(err_rel),
    "Spearman": spearmanr(y_true_log, y_pred_log).statistic,
}

```

```

RESULTADOS = []

```

```

def registra(*args, **kwargs):
    r = evalua(*args, **kwargs)
    RESULTADOS.append(r)
    print(f" {r['conjunto']:<6} MAE_log {r['MAE_log']:.4f} | R2 {r['R2_log']:.4f} | "
          f"MAE {r['MAE_eur']/1e6:>6.2f} M€ | MAPE {r['MAPE_%']:>6.1f}% | "
          f"rho {r['Spearman']:.4f}")
    return r

```

```

print("Funciones de evaluación definidas.")

```

```

## 3.3. Demostración empírica del sesgo de la partición aleatoria

```

Antes de entrenar los modelos definitivos se cuantifica el efecto de emplear una partición incorrecta. Se ajusta el mismo algoritmo con la misma configuración bajo dos esquemas: partición aleatoria por filas frente a partición temporal con agrupación por jugador.

La diferencia entre ambos resultados cuantifica el optimismo que introduciría una evaluación mal diseñada. Su magnitud depende del solapamiento de jugadores entre particiones, por lo que el resultado obtenido —cualquiera que sea— es en sí mismo informativo sobre la naturaleza del problema.

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

```

```

def prep_arboles(num, cat):
    return ColumnTransformer([
        ("num", SimpleImputer(strategy="median"), num),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="constant", fill_value="desconocido")),
                          ("enc", OrdinalEncoder(handle_unknown="use_encoded_value",
                                                  unknown_value=-1))])), cat),
    ], remainder="drop")

```

```

esp = ESPECIFICACIONES["principal"]
COLS = esp["num"] + esp["cat"]
rf_demo = lambda: RandomForestRegressor(n_estimators=300, min_samples_leaf=3,

```

```

n_jobs=-1, random_state=SEED)

# --- Esquema A: partición aleatoria por filas (INCORRECTO) -----
Xa, ya = d[COLS], d[TARGET]
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(Xa, ya, test_size=.2, random_state=SEED)
pipe_a = Pipeline([("prep", prep_arboles(esp["num"], esp["cat"])), ("m", rf_demo())])
pipe_a.fit(Xa_tr, ya_tr)
r2_aleatorio = r2_score(ya_te, pipe_a.predict(Xa_te))

# --- Esquema B: partición temporal + agrupación (CORRECTO) -----
pipe_b = Pipeline([("prep", prep_arboles(esp["num"], esp["cat"])), ("m", rf_demo())])
pipe_b.fit(tr[COLS], tr[TARGET])
r2_temporal = r2_score(te[TARGET], pipe_b.predict(te[COLS]))

print("=== EFECTO DEL DISEÑO DE VALIDACIÓN (Random Forest, especificación principal)
===")
print(f" Partición aleatoria por filas R² = {r2_aleatorio:.4f}")
print(f" Partición temporal + agrupación R² = {r2_temporal:.4f}")
print(f"\n Optimismo introducido: {r2_aleatorio - r2_temporal:+.4f} "
      f"f'({100*(r2_aleatorio-r2_temporal)/max(r2_temporal,1e-9):+.1f}%)")
print("\nTodas las métricas de este trabajo se reportan bajo el esquema correcto.")
---
```

4. Modelos

Se entrenan cinco técnicas en progresión de complejidad. La comparación no busca únicamente identificar el mejor resultado, sino documentar las fortalezas y debilidades de cada familia sobre este problema concreto.

#	Modelo	Papel en el análisis
1	Persistencia	Baseline realista, no trivial
2	Regresión lineal con splines	Referencia interpretable, coeficientes con lectura económica
3	Random Forest	Primer modelo no lineal, con ajuste de hiperparámetros
4	CatBoost	Modelo final, con ajuste y estimación de intervalos
5	Red neuronal con embeddings	Contraste con aprendizaje profundo

4.1. Modelo 1 — Persistencia

El baseline no es un predictor constante, sino la regla que un observador informado aplicaría: un jugador valdrá aproximadamente lo mismo que el año anterior.

```

from sklearn.dummy import DummyRegressor

print("=== MODELO 1a: predictor constante (mediana) ===")
dum = DummyRegressor(strategy="median").fit(tr[COLS], tr[TARGET])
for nom, cj in [("valid", va), ("test", te)]:
    registra("Constante (mediana)", "—", nom, cj[TARGET],
            dum.predict(cj[COLS]), cj["indice_mercado"], cj["target_valor"])

print("\n=== MODELO 1b: persistencia (V_{t+1} = V_t) ===")
for nom, cj in [("valid", va), ("test", te)]:
    m = cj["log_mv_prev"].notna()
    sub_ = cj[m]
```

```
registra("Persistencia", "persistencia", nom, sub_[TARGET],
        sub_["log_mv_prev"].values, sub_["indice_mercado"], sub_["target_valor"])
## 4.2. Modelo 2 — Regresión lineal con splines de edad
```

La edad se modela mediante splines cúbicos naturales, lo que permite capturar la curva de carrera —no monótona, con un máximo intermedio— dentro de un marco lineal.

```
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Variables continuas cuya relación con el objetivo es plausiblemente multiplicativa
LOG_VARS = ["minutos", "club_valor_medio", "club_aforo"]

def prepara_ols(dfx):
    x = dfx.copy()
    for c in LOG_VARS:
        x[f"log_{c}"] = np.log1p(x[c])
    return x

tr_ols, va_ols, te_ols = prepara_ols(tr), prepara_ols(va), prepara_ols(te)

TERMINOS = ["log_minutos", "goles_90", "asistencias_90", "amarillas_90",
            "ratio_titular", "min_por_partido", "capitanias",
            "partidos_euro", "n_competiciones",
            "club_ppp", "club_rank", "log_club_valor_medio", "log_club_aforo",
            "height_in_cm"]

# Spline cúbico natural sobre la edad; si 'cr' no está disponible, se usa 'bs'
for spline in ["cr(edad, df=5)", "bs(edad, df=5)"]:
    formula = f"{TARGET} ~ {spline} + " + " + ".join(TERMINOS) + \
        " + C(position) + C(foot) + C(liga)"
    try:
        ols = smf.ols(formula, data=tr_ols).fit()
        print(f"Spline empleado: {spline}")
        break
    except Exception as e:
        print(f" {spline} no disponible ({type(e).__name__})")

print(f"\nR² ajustado (train): {ols.rsquared_adj:.4f} | AIC: {ols.aic:,.0f}")
print(f"Observaciones: {int(ols.nobs):,}")

print("\n=== COEFICIENTES SIGNIFICATIVOS (p < 0.01), sin splines ni dummies ===")
res = pd.DataFrame({"coef": ols.params, "p": ols.pvalues})
res = res[~res.index.str.contains(r"edad|Intercept|C\(")]
print(res[res["p"] < .01].sort_values("coef", key=abs, ascending=False).round(4).to_string())

print("\n=== VIF (multicolinealidad) ===")
Xv = tr_ols[[c for c in TERMINOS]].dropna()
vif = pd.DataFrame({"variable": Xv.columns,
                    "VIF": [variance_inflation_factor(Xv.values, i)
                           for i in range(Xv.shape[1])])})
print(vif.sort_values("VIF", ascending=False).round(2).to_string(index=False))
```

```

print("\nVIF > 10 indica multicolinealidad relevante.")
# Corrección de Duan sobre los residuos de entrenamiento
factor_ols = duan(ols.resid.values)
print(f"Factor de Duan (OLS): {factor_ols:.4f}")

print("\n=== MODELO 2: OLS con splines ===")
for nom, cj in [("valid", va_ols), ("test", te_ols)]:
    pred = ols.predict(cj)
    m = pred.notna()
    registra("OLS + splines", "principal", nom, cj.loc[m, TARGET], pred[m],
            cj.loc[m, "indice_mercado"], cj.loc[m, "target_valor"], factor_ols)
# Diagnóstico de residuos (Anexo)
fig, ax = plt.subplots(1, 3, figsize=(13, 3.5))

ax[0].scatter(ols.fittedvalues, ols.resid, s=3, alpha=.15)
ax[0].axhline(0, c="k", lw=1); ax[0].set_xlabel("Valores ajustados")
ax[0].set_ylabel("Residuos"); ax[0].set_title("Residuos vs ajustados")

import scipy.stats as st
st.probplot(ols.resid, dist="norm", plot=ax[1]); ax[1].set_title("Q-Q normal")

ax[2].hist(ols.resid, bins=50, color="#4C72B0"); ax[2].set_title("Distribución de residuos")
ax[2].set_xlabel("Residuo")

fig.tight_layout(); fig.savefig(FIGURES / "figA_diagnostico_ols.png", bbox_inches="tight")
plt.show()

from statsmodels.stats.diagnostic import het_breuschpagan
bp = het_breuschpagan(ols.resid, ols.model.exog)
print(f"Breusch-Pagan: estadístico {bp[0]:.1f}, p-valor {bp[1]:.3g}")
print("p < 0.05 indica heterocedasticidad: los errores estándar clásicos")
print("deben interpretarse con cautela.")
### 4.3. Modelo 3 — Random Forest

```

Primer modelo capaz de capturar relaciones no lineales e interacciones sin especificarlas. Los hiperparámetros se ajustan mediante búsqueda aleatoria con validación cruzada. Se entrena bajo las tres especificaciones para cuantificar la aportación de cada bloque de información.

```

PARAMS_RF = {
    "persistencia": {"n_estimators": 300, "min_samples_split": 2,
                     "min_samples_leaf": 4, "max_features": 0.6, "max_depth": 12},
    "principal": {"n_estimators": 500, "min_samples_split": 2,
                  "min_samples_leaf": 2, "max_features": 0.6, "max_depth": None},
    "robustez": {"n_estimators": 500, "min_samples_split": 2,
                 "min_samples_leaf": 2, "max_features": 0.6, "max_depth": 25},
}

```

```

modelos_rf = {}
for nombre_esp in ["persistencia", "principal", "robustez"]:
    print(f"\n=== RANDOM FOREST — especificación '{nombre_esp}' ===")
    esp = ESPECIFICACIONES[nombre_esp]

```

```
cols = esp["num"] + esp["cat"]
trx = tr.dropna(subset=esp["num"]) if nombre_esp != "principal" else tr
```

```
modelo = Pipeline([
    ("prep", prep_arboles(esp["num"], esp["cat"])),
    ("m", RandomForestRegressor(**PARAMS_RF[nombre_esp],
                                n_jobs=-1, random_state=SEED)))
```

```
t0 = time.time()
modelo.fit(trx[cols], trx[TARGET])
print(f" entrenado en {time.time()-t0:.0f}s")
```

```
factor = duan((trx[TARGET] - modelo.predict(trx[cols])).values)
modelos_rf[nombre_esp] = (modelo, cols, factor)
```

```
for nom, cj in [("valid", va), ("test", te)]:
    cjk = cj
    registra("Random Forest", nombre_esp, nom, cjk[TARGET],
            modelo.predict(cjk[cols]), cjk["indice_mercado"],
            cjk["target_valor"], factor)
```

4.4. Modelo 4 — CatBoost

Se selecciona sobre otras implementaciones de *gradient boosting* por dos motivos: trata las variables categóricas de forma nativa mediante *ordered target statistics*, evitando la expansión que produciría una codificación one-hot y el sobreajuste asociado a una codificación por objetivo construida manualmente; y su esquema de *ordered boosting* reduce el sesgo de predicción en conjuntos de tamaño moderado como este.

```
from catboost import CatBoostRegressor, Pool
```

```
def pools(espec_nombre):
    esp = ESPECIFICACIONES[espec_nombre]
    cols = esp["num"] + esp["cat"]
    idx = [cols.index(c) for c in esp["cat"]]
    # No se descartan filas por ausencia de valoracion previa: CatBoost trata
    # los nulos de forma nativa y asi las tres especificaciones se evaluan
    # sobre exactamente el mismo conjunto de prueba.
    trx, vax, tex = (x.reset_index(drop=True) for x in (tr, va, te))
    P = lambda x: Pool(x[cols], x[TARGET], cat_features=idx)
    return P(trx), P(vax), P(tex), cols, esp, trx, vax, tex
```

```
# Hiperparámetros obtenidos en una búsqueda aleatoria previa (20 combinaciones,
# validación cruzada de 3 pliegues). Se reutilizan por el mismo motivo indicado
# para Random Forest.
```

```
PARAMS_CB = {
    "persistencia": {"subsample": 0.7, "random_strength": 6, "depth": 10,
                     "learning_rate": 0.05, "l2_leaf_reg": 3},
    "principal": {"subsample": 1, "random_strength": 3, "depth": 8,
                  "learning_rate": 0.05, "l2_leaf_reg": 1},
    "robustez": {"subsample": 0.85, "random_strength": 3, "depth": 8,
                 "learning_rate": 0.02, "l2_leaf_reg": 3},
}
```

```

modelos_cb = {}
for nombre_esp in ["persistencia", "principal", "robustez"]:
    print(f"\n=== CATBOOST — especificación '{nombre_esp}' ===")
    p_tr, p_va, p_te, cols, esp, trx, vax, tex = pools(nombre_esp)

    modelo = CatBoostRegressor(**PARAMS_CB[nombre_esp], iterations=3000,
                                loss_function="RMSE", eval_metric="MAE",
                                random_seed=SEED, od_type="Iter", od_wait=100,
                                verbose=0)

    t0 = time.time()
    modelo.fit(p_tr, eval_set=p_va, use_best_model=True)
    print(f" entrenado en {time.time()-t0:.0f}s | "
          f"iteraciones óptimas: {modelo.get_best_iteration()}")

    factor = duan((trx[TARGET] - modelo.predict(p_tr)).values)
    modelos_cb[nombre_esp] = (modelo, cols, esp, factor)

    for nom, cj, pl_ in [("valid", vax, p_va), ("test", tex, p_te)]:
        registra("CatBoost", nombre_esp, nom, cj[TARGET], modelo.predict(pl_),
                cj["indice_mercado"], cj["target_valor"], factor)
#### Intervalos de predicción

```

Una valoración expresada como un único número transmite una precisión que el modelo no posee.

Se entrenan tres modelos con función de pérdida cuantílica —cuantiles 0,1, 0,5 y 0,9— que permiten acompañar cada estimación de un intervalo con una cobertura nominal del 80 %.

La cobertura empírica sobre el conjunto de prueba verifica si el intervalo cumple lo que promete.

```

p_tr, p_va, p_te, cols, esp, trx, vax, tex = pools("principal")
assert "log_mv_prev" not in cols, "Los cuantiles deben usar la especificacion principal"

```

```

cuantiles = {}
for q in [0.1, 0.5, 0.9]:
    m = CatBoostRegressor(iterations=1200, loss_function=f"Quantile:alpha={q}",
                            depth=6, learning_rate=0.05, random_seed=SEED, verbose=0)
    m.fit(p_tr, eval_set=p_va, use_best_model=True)
    cuantiles[q] = m
    print(f" cuantil {q}: entrenado")

```

```

lo, med, hi = (cuantiles[q].predict(p_te) for q in [0.1, 0.5, 0.9])

```

Los tres cuantiles se entrenan por separado y pueden cruzarse.

Se fuerza la monotonía, práctica habitual en regresión cuantílica.

```

cruces = (lo > med) | (med > hi)
print(f"Cruces de cuantiles corregidos: {cruces.sum()} de {len(lo)} ({cruces.mean():.1%}")
lo, med, hi = np.sort(np.vstack([lo, med, hi]), axis=0)

```

```

dentro = ((tex[TARGET] >= lo) & (tex[TARGET] <= hi)).mean()
print(f"\nCobertura empírica del intervalo 80%: {dentro:.1%}")
print(f"Amplitud mediana (escala log): {np.median(hi - lo):.3f}")

```

```

lo_e = a_euros(lo, tex["indice_mercado"])
med_e = a_euros(med, tex["indice_mercado"])
hi_e = a_euros(hi, tex["indice_mercado"])
print(f"Amplitud mediana en euros: {np.median(hi_e - lo_e)/1e6:.2f} M€")
orden = np.argsort(tex["target_valor"].values)
pos = orden[np.linspace(0, len(orden)-1, 6).astype(int)]

ej = tex.iloc[pos].reset_index(drop=True)
ej["inf_M€"] = np.asarray(lo_e)[pos] / 1e6
ej["pred_M€"] = np.asarray(med_e)[pos] / 1e6
ej["sup_M€"] = np.asarray(hi_e)[pos] / 1e6
ej["real_M€"] = ej["target_valor"].values / 1e6
ej["acierta"] = (ej["real_M€"] >= ej["inf_M€"]) & (ej["real_M€"] <= ej["sup_M€"])

print("\nEjemplos de predicción con intervalo:")
print(ej[["season", "edad", "position", "liga", "inf_M€", "pred_M€", "sup_M€",
        "real_M€", "acierta"]].round(2).to_string(index=False))
## 4.5. Modelo 5 — Red neuronal con embeddings de entidades

```

Se anticipa que este modelo no obtendrá el mejor resultado. En problemas tabulares de tamaño moderado, los métodos de *boosting* superan habitualmente a las redes neuronales. Su inclusión responde a un motivo: documentar empíricamente esa diferencia sobre el problema concreto.

```

try:
    import torch, torch.nn as nn
    from torch.utils.data import TensorDataset, DataLoader
    TORCH = True
    torch.manual_seed(SEED)
except ImportError:
    TORCH = False
    print("PyTorch no disponible: se omite el modelo 5.")
if TORCH:
    from sklearn.preprocessing import StandardScaler

    esp = ESPECIFICACIONES["principal"]
    NUMc, CATc = esp["num"], esp["cat"]

    # Codificación de categóricas a índices, ajustada solo con train
    mapas = {c: {v: i for i, v in enumerate(sorted(tr[c].unique()))} for c in CATc}
    cards = {c: len(m) + 1 for c, m in mapas.items()}    # +1 para desconocidos

    def idx_cat(dfx):
        return np.stack([dfx[c].map(mapas[c]).fillna(len(mapas[c])).astype(int).values
                        for c in CATc], axis=1)

    imp = tr[NUMc].median()
    sc = StandardScaler().fit(tr[NUMc].fillna(imp))
    Xn = lambda dfx: sc.transform(dfx[NUMc].fillna(imp)).astype(np.float32)

    def tensores(dfx):

```

```

return (torch.tensor(Xn(dfX)), torch.tensor(idx_cat(dfX)),
        torch.tensor(dfX[TARGET].values, dtype=torch.float32))

```

```

Xtr_n, Xtr_c, ytr = tensores(tr)
Xva_n, Xva_c, yva = tensores(va)
Xte_n, Xte_c, yte = tensores(te)

```

```

class MLPEmb(nn.Module):
    def __init__(self, n_num, cards, oculto=(128, 64), p=0.25):
        super().__init__()
        self.emb = nn.ModuleList([
            nn.Embedding(card, min(16, (card + 1) // 2)) for card in cards.values()])
        dim_emb = sum(e.embedding_dim for e in self.emb)
        capas, entrada = [], n_num + dim_emb
        for h in oculto:
            capas += [nn.Linear(entrada, h), nn.BatchNorm1d(h), nn.ReLU(), nn.Dropout(p)]
            entrada = h
        capas.append(nn.Linear(entrada, 1))
        self.red = nn.Sequential(*capas)

    def forward(self, xn, xc):
        e = torch.cat([emb(xc[:, i]) for i, emb in enumerate(self.emb)], dim=1)
        return self.red(torch.cat([xn, e], dim=1)).squeeze(-1)

```

```

red = MLPEmb(len(NUMc), cards)
opt = torch.optim.AdamW(red.parameters(), lr=2e-3, weight_decay=1e-4)
perdida = nn.L1Loss()
loader = DataLoader(TensorDataset(Xtr_n, Xtr_c, ytr), batch_size=256, shuffle=True)

```

```

mejor, paciencia, espera = np.inf, 25, 0
for epoca in range(300):
    red.train()
    for bn, bc, by in loader:
        opt.zero_grad(); perdida(red(bn, bc), by).backward(); opt.step()
    red.eval()
    with torch.no_grad():
        mae_va = perdida(red(Xva_n, Xva_c), yva).item()
    if mae_va < mejor - 1e-4:
        mejor, espera = mae_va, 0
        estado = {k: v.clone() for k, v in red.state_dict().items()}
    else:
        espera += 1
        if espera >= paciencia:
            print(f" parada temprana en época {epoca} | mejor MAE(valid) = {mejor:.4f}")
            break
    red.load_state_dict(estado)
if TORCH:
    red.eval()
    with torch.no_grad():
        p_tr_ = red(Xtr_n, Xtr_c).numpy()
        p_va_ = red(Xva_n, Xva_c).numpy()

```

```

p_te_ = red(Xte_n, Xte_c).numpy()

factor_nn = duan(tr[TARGET].values - p_tr_)
print("=== MODELO 5: red neuronal con embeddings ===")
for nom, cj, pr in [("valid", va, p_va_), ("test", te, p_te_)]:
    registra("MLP + embeddings", "principal", nom, cj[TARGET], pr,
            cj["indice_mercado"], cj["target_valor"], factor_nn)
if TORCH:
    # Los embeddings aprendidos son un resultado interpretable en sí mismos
    i_liga = CATc.index("liga")
    E = red.emb[i_liga].weight.detach().numpy()
    etiquetas = list(sorted(tr["liga"].unique())) + ["desconocido"]

    from sklearn.decomposition import PCA
    P = PCA(n_components=2, random_state=SEED).fit_transform(E)

    fig, ax = plt.subplots(figsize=(5.5, 4.5))
    ax.scatter(P[:, 0], P[:, 1], s=60, c="#4C72B0")
    for (x, y), t in zip(P, etiquetas):
        ax.annotate(t, (x, y), fontsize=9, xytext=(4, 4), textcoords="offset points")
    ax.set_title("Embeddings aprendidos de la variable 'liga' (PCA)")
    ax.set_xlabel("Componente 1"); ax.set_ylabel("Componente 2")
    fig.tight_layout(); fig.savefig(FIGURES / "fig6_embeddings_liga.png", bbox_inches="tight")
    plt.show()

    print("Distancias entre ligas en el espacio latente:")
    from scipy.spatial.distance import pdist, squareform
    D = pd.DataFrame(squareform(pdist(E)), index=etiquetas, columns=etiquetas)
    print(D.round(2).to_string())

```

5. Resultados

5.1. Tabla comparativa

La lectura debe hacerse ****por especificación****. Comparar un modelo que incorpora la valoración previa con otro que no lo hace no aporta información, ya que resuelven problemas de dificultad distinta.

```

R = pd.DataFrame(RESPONSES)
R.to_csv(PROCESSED / "resultados_modelos.csv", index=False, encoding="utf-8")

```

```

for cj in ["valid", "test"]:
    print(f"\n{'='*95}\nCONJUNTO: {cj.upper()}\n{'='*95}")
    t = (R[R.conjunto == cj]
        .sort_values(["espec", "MAE_log"])
        [["espec", "modelo", "MAE_log", "RMSE_log", "R2_log",
            "MAE_eur", "MedAE_eur", "MAPE_%", "Spearman"]])
    t["MAE_eur"] = (t["MAE_eur"]/1e6).round(2)
    t["MedAE_eur"] = (t["MedAE_eur"]/1e6).round(2)
    print(t.rename(columns={"MAE_eur": "MAE_M€", "MedAE_eur": "MedAE_M€"}).
        .round(4).to_string(index=False))

# Comparación gráfica sobre el conjunto de prueba
tt = R[(R.conjunto == "test") & (R.espec != "—")]

```

```

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
for i, (met, tit) in enumerate([("MAE_log", "MAE (escala log) — menor es mejor"),
                               ("R2_log", "R² (escala log) — mayor es mejor")]):
    piv = tt.pivot_table(index="modelo", columns="espec", values=met)
    piv.plot(kind="barh", ax=ax[i], width=.75)
    ax[i].set_title(tit); ax[i].set_xlabel(met); ax[i].set_ylabel("")
    ax[i].legend(title="Especificación", fontsize=7)

fig.suptitle("Comparación de modelos sobre el conjunto de prueba", y=1.03)
fig.tight_layout(); fig.savefig(FIGURES / "fig7_comparativa.png", bbox_inches="tight")
plt.show()

```

5.2. Análisis del error por segmento

El error medio oculta comportamientos muy distintos según el tramo de valor. Se examina dónde falla el modelo, cuestión que resulta determinante para el notebook 3: si el modelo infravalora sistemáticamente a los jugadores más caros, los residuos negativos de ese tramo reflejarán un sesgo del modelo y no una oportunidad de mercado.

```

modelo_f, cols_f, esp_f, factor_f = modelos_cb["principal"]
p_te_f = Pool(te[cols_f], te[TARGET], cat_features=[cols_f.index(c) for c in esp_f["cat"]])

```

```

te_an = te.copy()
te_an["pred_log"] = modelo_f.predict(p_te_f)
te_an["pred_eur"] = a_euros(te_an["pred_log"], te_an["indice_mercado"], factor_f)
te_an["residuo"] = te_an[TARGET] - te_an["pred_log"]
te_an["err_abs"] = te_an["residuo"].abs()
te_an["err_rel"] = (te_an["pred_eur"] - te_an["target_valor"]).abs() / te_an["target_valor"]

```

```

te_an["tramo"] = pd.qcut(te_an["target_valor"], 5,
                        labels=["Muy bajo", "Bajo", "Medio", "Alto", "Muy alto"])

```

```

print("=== ERROR POR TRAMO DE VALOR ===")
print(te_an.groupby("tramo", observed=True)
      .agg(n=("residuo", "size"),
           valor_mediano_M=("target_valor", lambda s: s.median()/1e6),
           MAE_log=("err_abs", "mean"),
           sesgo_medio=("residuo", "mean"),
           MAPE_pct=("err_rel", lambda s: 100*s.mean()))
      .round(3).to_string())

```

```

print("\nUn sesgo medio positivo indica INFRAVALORACIÓN sistemática del modelo.")

```

```

print("\n=== ERROR POR POSICIÓN ===")
print(te_an.groupby("position").agg(n=("residuo", "size"), MAE_log=("err_abs", "mean"),
                                   sesgo=("residuo", "mean")).round(3).to_string())

```

```

print("\n=== ERROR POR LIGA ===")
print(te_an.groupby("liga").agg(n=("residuo", "size"), MAE_log=("err_abs", "mean"),
                                sesgo=("residuo", "mean")).round(3).to_string())

```

```

print("\n=== ERROR POR TRAMO DE EDAD ===")

```

```

te_an["tramo_edad"] = pd.cut(te_an["edad"], [15,21,25,29,33,50],
                             labels=["<21", "21-25", "25-29", "29-33", ">33"])
print(te_an.groupby("tramo_edad", observed=True)
      .agg(n=("residuo", "size"), MAE_log=("err_abs", "mean"),
           sesgo=("residuo", "mean")).round(3).to_string())
INTERIM = PROJ / "data" / "interim"
app_ = pd.read_parquet(INTERIM / "appearances.parquet")
gam_ = pd.read_parquet(INTERIM / "games.parquet")[["game_id", "season"]]
apps = app_.merge(gam_, on="game_id", how="left")

# 1. Apariciones GB1 en 2023?
ing = apps[(apps.season == 2023) & (apps.competition_id == "GB1")]
print(f"GB1 2023: {len(ing):,} apariciones | {ing.player_id.nunique():,} jugadores")

# 2. Cobertura GB1 por temporada
print("\nApariciones por liga y temporada:")
print(pd.crosstab(apps[apps.season.between(2016, 2023)][["season"],
                  apps[apps.season.between(2016, 2023)][["competition_id"]
                  ]][["GB1", "ES1", "IT1", "L1", "FR1"]].to_string())

# 3. Esos jugadores, a que liga se les asigna en dataset final?
jug = set(ing.player_id)
sub = a[(a.season == 2023) & (a.player_id.isin(jug))]
print(f"\nJugadores GB1 2023 en dataset final: {len(sub):,}")
print(sub["liga"].value_counts(dropna=False).to_string() if len(sub) else " ninguno")
fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))

ax[0].scatter(te_an["pred_log"], te_an[TARGET], s=5, alpha=.25)
lim = [te_an[TARGET].min(), te_an[TARGET].max()]
ax[0].plot(lim, lim, "k--", lw=1)
ax[0].set_xlabel("Predicho (log)"); ax[0].set_ylabel("Real (log)")
ax[0].set_title("Predicho frente a real")

ax[1].scatter(te_an["pred_log"], te_an["residuo"], s=5, alpha=.25, color="#C44E52")
ax[1].axhline(0, c="k", lw=1)
ax[1].set_xlabel("Predicho (log)"); ax[1].set_ylabel("Residuo")
ax[1].set_title("Residuos frente a predicho")

sns.boxplot(data=te_an, x="tramo", y="residuo", ax=ax[2], showfliers=False, palette="RdBu")
ax[2].axhline(0, c="k", lw=1)
ax[2].set_xlabel("Tramo de valor real"); ax[2].set_ylabel("Residuo")
ax[2].set_title("Sesgo por tramo")

fig.suptitle("Análisis del error del modelo final (conjunto de prueba)", y=1.03)
fig.tight_layout(); fig.savefig(FIGURES / "fig8_analisis_error.png", bbox_inches="tight")
plt.show()
## 5.3. Selección del modelo final y serialización

```

Se conserva el modelo junto con toda la información necesaria para reproducir una predicción: las columnas de entrada en su orden, los índices de las variables categóricas, el factor de corrección de Duan y los modelos cuantílicos que proporcionan el intervalo.

Guardar el modelo aislado sería insuficiente: sin el factor de Duan y sin el índice de mercado de la temporada, la predicción no puede convertirse a euros.

```
import joblib
```

```
artefacto = {
    "modelo":      modelos_cb["principal"][0],
    "columnas":    cols_f,
    "cat_features": [cols_f.index(c) for c in esp_f["cat"]],
    "num_features": esp_f["num"],
    "cat_nombres": esp_f["cat"],
    "factor_duan": factor_f,
    "cuantiles":   cuantiles,
    "indice_mercado": a.groupby("season")["target_valor"].median().to_dict(),
    "target":      TARGET,
    "seed":        SEED,
    "especificacion": "principal",
}
joblib.dump(artefacto, MODELS / "modelo_final.joblib")
print("Modelo serializado en", MODELS / "modelo_final.joblib")
```

```
# Los residuos alimentan el notebook 3
```

```
te_an.to_parquet(PROCESSED / "predicciones_test.parquet", index=False)
print("Predicciones de test guardadas para el notebook 3.")
```

```
joblib.dump({k: (v[0], v[1], v[2]) for k, v in modelos_rf.items()},
            MODELS / "modelos_rf.joblib")
print("Modelos Random Forest guardados.")
---
```

```
## Cierre
```

```
**Ficheros generados:**
```

```
| Fichero | Contenido |
|---|---|
| `models/modelo_final.joblib` | Modelo, metadatos, factor de Duan y modelos cuantílicos |
| `models/modelos_rf.joblib` | Random Forest por especificación |
| `data/processed/resultados_modelos.csv` | Tabla comparativa completa |
| `data/processed/predicciones_test.parquet` | Predicciones y residuos sobre prueba |
| `reports/figures/*.png` | Figuras para la memoria |
```

```
**Continúa en `03_interpretabilidad.ipynb`: interpretación mediante SHAP e identificación de jugadores infravalorados.
```

ANEXO F: CÓDIGO DE NOTEBOOK

03_INTERRETABILIDAD.IPYNB

Estimación del valor de mercado de futbolistas profesionales

Notebook 3 — Interpretabilidad y aplicación de negocio

****Trabajo Fin de Máster**** · Máster en Big Data, Data Science e Inteligencia Artificial · Universidad Complutense de Madrid

Qué hace este notebook

****Consume****	`models/modelo\_final.joblib`, `data/processed/predicciones\_test.parquet`
****Produce****	Ranking de jugadores infravalorados y función de inferencia
****Alimenta****	Capítulos 6 (Interpretabilidad), 7 (Aplicación de negocio) y 8 (Empaquetado)

Estructura

1. ****Interpretabilidad global**** — qué determina el valor de un futbolista
2. ****Efectos no lineales**** — cómo influye cada factor a lo largo de su rango
3. ****Interpretabilidad local**** — tres jugadores explicados en detalle
4. ****Identificación de infravalorados**** — añadido como análisis de resultados
5. ****Empaquetado**** — función de inferencia sobre nuevos datos

0. Configuración y carga de artefactos

```
from pathlib import Path
import warnings, joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import shap
```

```
warnings.filterwarnings("ignore")
```

```
SEED = 42
```

```
np.random.seed(SEED)
```

```
PROJ = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
```

```
INTERIM = PROJ / "data" / "interim"
```

```
PROCESSED = PROJ / "data" / "processed"
```

```
MODELS = PROJ / "models"
```

```
FIGURES = PROJ / "reports" / "figures"
```

```
FIGURES.mkdir(parents=True, exist_ok=True)
```

```

sns.set_theme(style="whitegrid")
plt.rcParams.update({"figure.dpi": 110, "font.size": 9})
pd.set_option("display.max_columns", 60)
pd.set_option("display.width", 150)
pd.set_option("display.float_format", lambda x: f"{x:.,3f}")

# --- Artefactos del notebook 2 -----
art = joblib.load(MODELS / "modelo_final.joblib")
modelo = art["modelo"]
COLS = art["columnas"]
CATS = art["cat_nombres"]
IDXCAT = art["cat_features"]
DUAN = art["factor_duan"]
CUANT = art["cuantiles"]
INDICE = art["indice_mercado"]

te = pd.read_parquet(PROCESSED / "predicciones_test.parquet")
a = pd.read_parquet(PROCESSED / "dataset_analitico.parquet")

# Nombres de jugador y club, solo para legibilidad de los ejemplos
players = pd.read_parquet(INTERIM / "players.parquet")[["player_id", "name"]]
clubs = pd.read_parquet(INTERIM / "clubs.parquet")[["club_id", "name"]] \
        .rename(columns={"name": "club"})
te = te.merge(players, on="player_id", how="left").merge(clubs, on="club_id", how="left")

print(f"Conjunto de prueba: {len(te):,} filas | {te['player_id'].nunique():,} jugadores")
print(f"Variables del modelo: {len(COLS)} ({len(CATS)} categóricas)")
print(f"Factor de Duan: {DUAN:.4f}")
%load_ext watermark
%watermark -v -p pandas,numpy,catboost,shap
---
```

1. Interpretabilidad global

Los valores SHAP reparten la predicción de cada observación entre las variables que intervienen, de forma que la suma de contribuciones más el valor esperado reproduce exactamente la predicción. En modelos de árboles el cálculo es **exacto**, no aproximado.

Se emplea la implementación nativa de CatBoost, que respeta el tratamiento interno de las variables categóricas.

```
from catboost import Pool
```

```
pool_te = Pool(te[COLS], te["y_log_defl"], cat_features=IDXCAT)
```

ShapValues devuelve n_variables + 1 columnas; la última es el valor esperado

```
sv = modelo.get_feature_importance(pool_te, type="ShapValues")
```

```
shap_val = sv[:, :-1]
```

```
valor_esp = sv[0, -1]
```

```
expl = shap.Explanation(values=shap_val,
                      base_values=np.repeat(valor_esp, len(te)),
                      data=te[COLS].values,
```

```

feature_names=COLS)

print(f"Matriz SHAP: {shap_val.shape}")
print(f"Valor esperado (log): {valor_esp:.4f} -> "
      f"{np.expm1(valor_esp):.2f} veces la mediana de mercado")

# Comprobacion: SHAP debe reconstruir la prediccion
recon = shap_val.sum(axis=1) + valor_esp
print(f"Error máximo de reconstrucción: {np.abs(recon - te['pred_log']).max():.2e}")
### 1.1. Importancia global de las variables
imp = (pd.DataFrame({"variable": COLS,
                    "shap_medio_abs": np.abs(shap_val).mean(axis=0)}
        .sort_values("shap_medio_abs", ascending=False, ignore_index=True))
imp["%_del_total"] = (100 * imp["shap_medio_abs"] / imp["shap_medio_abs"].sum()).round(1)

print("=== IMPORTANCIA GLOBAL (media del valor absoluto SHAP) ===")
print(imp.head(20).to_string(index=False))

fig, ax = plt.subplots(figsize=(7, 6))
top = imp.head(18).iloc[::-1]
ax.barh(top["variable"], top["shap_medio_abs"], color="#4C72B0")
ax.set_xlabel("Media de |valor SHAP| (escala logarítmica)")
ax.set_title("Importancia global de las variables")
fig.tight_layout(); fig.savefig(FIGURES / "fig9_shap_importancia.png", bbox_inches="tight")
plt.show()

# Diagrama de dispersión SHAP: dirección y magnitud del efecto
fig = plt.figure(figsize=(8, 7))
shap.summary_plot(shap_val, te[COLS], feature_names=COLS,
                  max_display=18, show=False, plot_size=None)
plt.title("Distribución de los valores SHAP por variable", fontsize=11)
plt.tight_layout(); plt.savefig(FIGURES / "fig10_shap_beeswarm.png", bbox_inches="tight")
plt.show()
### 1.2. Importancia agrupada por bloques

```

Varias variables miden aspectos próximos y SHAP reparte el crédito entre ellas, de modo que ninguna aparece como dominante por sí sola. Agrupar por bloques temáticos ofrece una lectura más fiel de qué **tipo** de información sostiene la predicción.

```

BLOQUES = {
    "Contexto de club": ["club_ppp", "club_dif_g", "club_rank", "club_valor_medio", "club_aforo", "liga"],
    "Rol y minutos":   ["minutos", "partidos", "min_por_partido", "ratio_titular", "capitanias",
                        "partidos_liga", "convocatorias"],
    "Producción":      ["goles", "asistencias", "goles_90", "asistencias_90", "ga_90"],
    "Perfil":          ["edad", "height_in_cm", "position", "sub_position", "foot"],
    "Competición":     ["partidos_euro", "n_competiciones", "jugo_europa"],
    "Disciplina":      ["amarillas", "rojas", "amarillas_90"],
}

```

```

filas = []
for bloque, vs in BLOQUES.items():
    idx = [COLS.index(v) for v in vs if v in COLS]
    if idx:

```

```

filas.append({"bloque": bloque, "n_vars": len(idx),
            "shap_medio_abs": np.abs(shap_val[:, idx]).sum(axis=1).mean()})

bl = pd.DataFrame(filas).sort_values("shap_medio_abs", ascending=False, ignore_index=True)
bl["%_del_total"] = (100 * bl["shap_medio_abs"] / bl["shap_medio_abs"].sum()).round(1)
print("=== IMPORTANCIA POR BLOQUE ===")
print(bl.to_string(index=False))

fig, ax = plt.subplots(figsize=(6.5, 3.5))
ax.barh(bl["bloque"].iloc[:-1], bl["shap_medio_abs"].iloc[:-1], color="#55A868")
ax.set_xlabel("Contribución media absoluta (log)")
ax.set_title("Aportación de cada bloque de información")
fig.tight_layout(); fig.savefig(FIGURES / "fig11_shap_bloques.png", bbox_inches="tight")
plt.show()

no_cubiertas = [c for c in COLS if not any(c in v for v in BLOQUES.values())]
print("\nVariables no asignadas a ningún bloque:", no_cubiertas if no_cubiertas else "ninguna")
---
```

2. Efectos no lineales

Los gráficos de dependencia muestran cómo varía la contribución de una variable a lo largo de su rango. El eje vertical está en escala logarítmica, de modo que el efecto se lee de forma multiplicativa sobre el valor.

```

def dependencia(var, ax, color="#4C72B0"):
    i = COLS.index(var)
    x, y = te[var].values, shap_val[:, i]
    ax.scatter(x, y, s=5, alpha=.25, color=color)
    ax.axhline(0, c="k", lw=.8, ls="--")
    # Tendencia suavizada por percentiles
    if np.issubdtype(te[var].dtype, np.number):
        q = pd.qcut(te[var], min(20, te[var].nunique()), duplicates="drop")
        t = pd.DataFrame({"x": x, "y": y, "q": q}).groupby("q", observed=True).median()
        ax.plot(t["x"], t["y"], color="#C44E52", lw=2)
    ax.set_xlabel(var); ax.set_ylabel("Contribución SHAP (log)")
```

```

VARS = [v for v in ["edad", "minutos", "club_valor_medio", "ratio_titular",
                  "ga_90", "club_rank"] if v in COLS]
```

```

fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, v in zip(axes.ravel(), VARS):
    dependencia(v, ax)
fig.suptitle("Efecto de cada variable sobre el valor estimado (escala logarítmica)", y=1.01)
fig.tight_layout(); fig.savefig(FIGURES / "fig12_dependencia.png", bbox_inches="tight")
plt.show()

# Lectura del efecto de la edad en términos multiplicativos
i = COLS.index("edad")
tabla = (pd.DataFrame({"edad": te["edad"].round(), "shap": shap_val[:, i]}
            .groupby("edad").agg(n=("shap", "size"), shap_medio=("shap", "mean"))))
tabla = tabla[tabla["n"] >= 20]
tabla["factor_multiplicativo"] = np.exp(tabla["shap_medio"]).round(3)
```

```

print("=== EFECTO DE LA EDAD ===")
print(tabla.round(3).to_string())
print("\nLectura: 'factor_multiplicativo' indica por cuánto multiplica la edad el valor")
print("estimado respecto a la media. Un 1,00 equivale a un efecto neutro.")
---
# 3. Interpretabilidad local y análisis de jugadores infravalorados
# Los modelos cuantílicos guardados esperan sus propias columnas.
COLS_Q = list(CUANT[0.5].feature_names_)
print("Columnas del modelo principal :", len(COLS))
print("Columnas del modelo cuantílico:", len(COLS_Q))
print("Diferencia:", set(COLS_Q) ^ set(COLS))

IDXCAT_Q = [COLS_Q.index(c) for c in CATS if c in COLS_Q]
pool_q = Pool(te[COLS_Q], cat_features=IDXCAT_Q)
# Intervalo de estimación para todo el conjunto de prueba
q_lo = np.asarray(CUANT[0.1].predict(pool_q))
q_md = np.asarray(CUANT[0.5].predict(pool_q))
q_hi = np.asarray(CUANT[0.9].predict(pool_q))
q_lo, q_md, q_hi = np.sort(np.vstack([q_lo, q_md, q_hi]), axis=0)

ind = te["indice_mercado"].values
te["est_inf"] = np.exp(q_lo) * DUAN * ind
te["est_sup"] = np.exp(q_hi) * DUAN * ind
te["ratio_est_real"] = te["pred_eur"] / te["target_valor"]

te["señal"] = np.log(te["ratio_est_real"])
te["infravalorado"] = (te["target_valor"] < te["est_inf"]) & (te["minutos"] >= 900)

COLS_RK = ["name", "club", "liga", "position", "edad", "partidos", "minutos",
           "goles", "asistencias", "target_valor", "pred_eur", "est_inf", "est_sup",
           "ratio_est_real", "señal"]

def formatea(d):
    x = d[COLS_RK].copy()
    for c in ["target_valor", "pred_eur", "est_inf", "est_sup"]:
        x[c] = (x[c]/1e6).round(2)
    return x.rename(columns={"target_valor": "real_M€", "pred_eur": "est_M€",
                             "est_inf": "inf_M€", "est_sup": "sup_M€",
                             "ratio_est_real": "ratio"}).round(2)

print("=== TOP 20 INFRAVALORADOS (temporada 2023/24) ===")
print(formatea(te[te["infravalorado"]].nlargest(20, "señal")).to_string(index=False))

print("\n\n=== TOP 15 SOBREALORADOS ===")
sobre = (te["target_valor"] > te["est_sup"]) & (te["minutos"] >= 900)
print(formatea(te[sobre].nsmallest(15, "señal")).to_string(index=False))
# Visualización: estimado frente a real, con los señalados destacados
fig, ax = plt.subplots(figsize=(7, 6))

resto = te[~te["infravalorado"]]
ax.scatter(resto["target_valor"]/1e6, resto["pred_eur"]/1e6,

```

```

        s=8, alpha=.25, color="grey", label="Resto")
sel = te[te["infravalorado"]]
ax.scatter(sel["target_valor"]/1e6, sel["pred_eur"]/1e6,
           s=18, alpha=.75, color="#C44E52", label="Señalados como infravalorados")

lim = [0.1, te["target_valor"].max()/1e6]
ax.plot(lim, lim, "k--", lw=1, label="Estimación = valor de mercado")
ax.set_xscale("log"); ax.set_yscale("log")
ax.set_xlabel("Valor de mercado real (M€)"); ax.set_ylabel("Valor estimado (M€)")
ax.set_title("Identificación de candidatos a infravaloración")
ax.legend(fontsize=8)
fig.tight_layout(); fig.savefig(FIGURES / "fig15_ranking.png", bbox_inches="tight")
plt.show()

# --- Caso de infravaloración: selección y descomposición SHAP -----
# Se restringe a jugadores con temporada completa para que las métricas
# agregadas sean estables y la discrepancia no responda a pocos minutos.
cand = (te[(te["target_valor"] < te["est_inf"]) & (te["minutos"] >= 1800)]
        .nlargest(5, "señal"))
print("=== CANDIDATOS CON TEMPORADA COMPLETA ===")
print(cand[["name", "club", "liga", "position", "edad", "partidos", "minutos",
            "goles", "asistencias", "target_valor", "pred_eur", "est_inf"]]
        .to_string(index=False))

# Caso seleccionado
NOMBRE = "Matteo Gabbia"
i = te.index[te["name"] == NOMBRE][0]
pos = te.index.get_loc(i)
f = te.loc[i]

print(f"\n{'='*70}\nCASO DE INFRAVALORACIÓN — {f['name']}")
print(f" {f['club']} ({f['liga']}) | {f['position']} | {f['edad']:.2f} años")
print(f" {f['partidos']:.0f} partidos, {f['minutos']:.0f} min, "
      f"{f['goles']:.0f} goles, {f['asistencias']:.0f} asistencias")
print(f" Tasación real {f['target_valor']/1e6:.2f} M€ | "
      f"estimación {f['pred_eur']/1e6:.2f} M€ | "
      f"intervalo 80% [{f['est_inf']/1e6:.2f} – {f['est_sup']/1e6:.2f}] M€")
print(f" f(x) = {f['pred_log']:.3f} | E[f(X)] = {valor_esp:.3f} | "
      f"residuo = {f['residuo']:.3f}")

# Gráfico de cascada
fig = plt.figure(figsize=(8, 5.5))
shap.plots.waterfall(expl[pos], max_display=12, show=False)
plt.title(f"Caso de infravaloración — {f['name']}", fontsize=10)
plt.tight_layout()
plt.savefig(FIGURES / "fig14_waterfall_infravalorado.png", bbox_inches="tight")
plt.show()

# Tabla de contribuciones, para redactar el apartado 6.2 de la memoria
contrib = (pd.DataFrame({"variable": COLS,
                        "valor": te.loc[i, COLS].values,
                        "shap": shap_val[pos]})

```

```

        .assign(abs_shap=lambda x: x["shap"].abs())
        .sort_values("abs_shap", ascending=False)
        .drop(columns="abs_shap"))
print("\n=== CONTRIBUCIONES SHAP ORDENADAS ===")
print(contrib.head(12).to_string(index=False))
print(f"\nSuma de contribuciones + valor esperado = "
      f"{shap_val[pos].sum() + valor_esp:.4f} (= f(x))")
---
```

4. Empaquetado: función de inferencia

La guía del máster valora que el modelo pueda recibir nuevos valores y devolver una predicción. Se implementa una función que encapsula el pipeline completo: recibe las estadísticas de un jugador-temporada y devuelve el valor estimado con su intervalo, aplicando internamente el factor de corrección de Duan y el índice de mercado de la temporada.

```

COLS_Q = list(CUANT[0.5].feature_names_)
IDXCAT_Q = [COLS_Q.index(c) for c in CATS if c in COLS_Q]
```

```
def estima_valor(datos: dict, season: int = None, verbose: bool = True) -> dict:
```

```
    # Estima el valor de mercado de un jugador-temporada.
```

```
    # Las variables ausentes se imputan con la mediana del conjunto.
```

```
    def construye(columnas, idxcat):
```

```
        fila = {}
```

```
        for c in columnas:
```

```
            if c in datos and pd.notna(datos[c]):
```

```
                fila[c] = datos[c]
```

```
            elif c in CATS:
```

```
                fila[c] = "desconocido"
```

```
            else:
```

```
                fila[c] = float(a[c].median())
```

```
    X = pd.DataFrame([fila])[columnas]
```

```
    for c in CATS:
```

```
        if c in X.columns:
```

```
            X[c] = X[c].astype(str)
```

```
    return Pool(X, cat_features=idxcat)
```

```
    ind = INDICE.get(season, np.median(list(INDICE.values())) if season \
```

```
        else np.median(list(INDICE.values()))
```

```
    log_pred = float(modelo.predict(construye(COLS, IDXCAT))[0])
```

```
    p_q = construye(COLS_Q, IDXCAT_Q)
```

```
    q = np.sort([float(CUANT[k].predict(p_q)[0]) for k in (0.1, 0.5, 0.9)])
```

```
    salida = {
```

```
        "valor_estimado_eur": float(np.exp(log_pred) * DUAN * ind),
```

```
        "intervalo_80_eur": (float(np.exp(q[0]) * DUAN * ind),
```

```
                             float(np.exp(q[2]) * DUAN * ind)),
```

```
        "indice_mercado": float(ind),
```

```
        "log_pred": log_pred,
```

```
    }
```

```
    if verbose:
```

```
        v = salida["valor_estimado_eur"] / 1e6
```

```

    lo, hi = (x/1e6 for x in salida["intervalo_80_eur"])
    print(f"Valor estimado: {v:,.2f} M€ (intervalo 80%: {lo:,.2f} – {hi:,.2f} M€)")
return salida

```

```

print("Función de inferencia definida.")
print(f"Modelo principal: {len(COLS)} vars | Cuantílicos: {len(COLS_Q)} vars")
# Prueba 1: caso real del conjunto de prueba (debe reproducir la predicción)
ref = te.iloc[0]
print(f"Caso real: {ref['name']} | {ref['club']} | {ref['edad']:.1f} años")
print(f"Valor de mercado observado: {ref['target_valor']/1e6:.2f} M€\n")
r1 = estima_valor({c: ref[c] for c in COLS}, season=int(ref["season"]))
print(f"\nPredicción almacenada en el notebook 2: {ref['pred_eur']/1e6:.2f} M€")
print(f"Diferencia: {abs(r1['valor_estimado_eur']-ref['pred_eur'])/1e6:.4f} M€")
# Prueba 2: jugador hipotético
print("=== Delantero joven, gran temporada en un club medio ===")
estima_valor({
    "edad": 22, "height_in_cm": 182, "position": "Attack",
    "sub_position": "Centre-Forward", "foot": "right", "liga": "ES1",
    "partidos": 34, "minutos": 2900, "goles": 18, "asistencias": 7,
    "amarillas": 3, "rojas": 0,
    "goles_90": 18/(2900/90), "asistencias_90": 7/(2900/90),
    "ga_90": 25/(2900/90), "amarillas_90": 3/(2900/90),
    "min_por_partido": 2900/34, "ratio_titular": 0.92, "capitanias": 0,
    "partidos_liga": 32, "partidos_euro": 0, "n_competiciones": 2, "jugo_europa": 0,
    "club_ppp": 1.35, "club_dif_g": 0.15, "club_rank": 8,
    "club_valor_medio": 8_000_000, "club_aforo": 40_000,
}, season=2023)

print("\n=== Mismo jugador, diez años mayor ===")
estima_valor({
    "edad": 32, "height_in_cm": 182, "position": "Attack",
    "sub_position": "Centre-Forward", "foot": "right", "liga": "ES1",
    "partidos": 34, "minutos": 2900, "goles": 18, "asistencias": 7,
    "amarillas": 3, "rojas": 0,
    "goles_90": 18/(2900/90), "asistencias_90": 7/(2900/90),
    "ga_90": 25/(2900/90), "amarillas_90": 3/(2900/90),
    "min_por_partido": 2900/34, "ratio_titular": 0.92, "capitanias": 0,
    "partidos_liga": 32, "partidos_euro": 0, "n_competiciones": 2, "jugo_europa": 0,
    "club_ppp": 1.35, "club_dif_g": 0.15, "club_rank": 8,
    "club_valor_medio": 8_000_000, "club_aforo": 40_000,
}, season=2023)
---
## Cierre

**Ficheros generados:**

```

| Fichero | Contenido |

|---|---

| `reports/figures/fig9` a `fig16` | Figuras de interpretabilidad y validación |